

BlueBottle: Fast and Robust Blockchains through Subsystem Specialization

Preston Vander Vos^{2,3}, Alberto Sonnino^{1,2}, Giorgos Tsimos⁴, Philipp Jovanovic^{1,2}, Lefteris Kokoris-Kogias¹

¹Mysten Labs; ²University College London; ³Circle; ⁴pod Network

Abstract

Blockchain consensus faces a trilemma of security, latency, and decentralization. High-throughput systems often require a reduction in decentralization or robustness against strong adversaries, while highly decentralized and secure systems tend to have lower performance. We present BlueBottle, a two-layer consensus architecture to tackle these challenges. The main finalization layer, which we call the core layer, is run in a medium-sized validator set. It is instantiated with our novel BB-Core consensus protocol. BB-Core exposes two client finality paths following the FlexibleBFT [41] paradigm. Fast-path clients that assume $n = 5f + 1$ experience 2δ -finality latency. Our experiments show that BB-Core reduces latency by 20–25% compared to Mysticeti. Checkpoint-path clients wait for a checkpoint certificate (4δ latency), which tolerates an additional $f_{AbC} < 2f$ alive-but-corrupt participants—achieving a total of 60% equivocation-fault safety. The key mechanism is a single-proposal-per-height rule that guarantees at most one valid checkpoint exists per slot height, even under heavy equivocation. The secondary finalization layer, BB-Guard, is run along a highly decentralized set of guard validators (or full nodes) and provides synchronous decentralized timestamping, proactive misbehavior detection, and a synchronous recovery path. When guard validators observe equivocations or liveness failures they disseminate evidence, agree on misbehaving parties for exclusion or slashing, and either restart the core protocol (for liveness violations) or select a canonical fork (for safety violations). Clients can choose to accept the synchrony assumption as their main assumption and withstand a $n = 2f + 1$ security level. BB-Guard can be deployed along any consensus protocol. In BlueBottle combining BB-Core with BB-Guard allows clients to choose their security level against their latency requirements.

PVLDB Reference Format:

Preston Vander Vos, Alberto Sonnino, Giorgos Tsimos, Philipp Jovanovic, Lefteris Kokoris-Kogias. Bluebottle: Fast and Robust Blockchains through Subsystem Specialization. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX **PVLDB Artifact Availability:**

The source code, data, and/or other artifacts have been made available at <https://github.com/phvv/mysticeti/tree/odontoceti>.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

1 Introduction

In currently deployed blockchains it is the protocol, not the client, that decides what finality means. A transaction is final once it clears a certain point within the security, latency, and decentralization parameter space that the protocol designer fixed in advance. Yet clients do not value these dimensions equally: a high-value settlement may demand the strongest safety and tolerate waiting, while a latency-sensitive payment prefers the fastest confirmation a weaker assumption allows. Existing systems make this choice for all clients alike.

FaB [42] showed that lowering the fault-tolerance threshold from $n = 3f + 1$ to $n = 5f + 1$ can reduce finality latency from 3δ to 2δ . Exploring such trade-offs has recently regained traction, with protocols such as Kudzu [50], Hydrangea [52], and Minimmit [21]. These protocols trade Byzantine resilience for a two-round commit latency, making them more fragile on their fast paths than their $3f + 1$ counterparts. To soften this, Kudzu and Hydrangea retain a secondary commit path alongside the fast one, whereas Minimmit forgoes it entirely. Crucially, though, the secondary paths of Kudzu and Hydrangea only tolerate additional *crash* faults, i.e., they optimize for more liveness, while assuming the very same Byzantine threshold for safety as the fast path. Both paths therefore commit under the same safety assumption and clients of these systems are offered no choice over the security–latency trade-off. If a client is unwilling to trust the lower resilience assumption, it has no stronger, higher-latency path to fall back on. This raises an interesting practical question:

Can we design a consensus protocol that provides clients with options in the security–latency–decentralization space, finalizing a transaction as soon as the assumptions that the client trusts hold?

This paper answers affirmatively by introducing BlueBottle, a novel dual-layer consensus architecture that synergistically combines the strengths of different consensus paradigms. Rather than resolving the security–latency–decentralization trilemma at design time on behalf of all clients, BlueBottle exposes it: each client selects the point it trusts and finalizes at the latency that choice affords. At BlueBottle’s core is a partially synchronous consensus protocol, BB-Core, operated by a medium-sized set of n_c core validators, that trades off failure tolerance ($n_c = 5f_c + 1$) for lower latency. It allows clients to decide on 2δ latency with $f_c < \frac{n_c}{5}$, or a 4δ latency with $f_c < \frac{n_c}{5}$ and an additional $f_{AbC} < \frac{2n_c}{5}$ where f_{AbC} can equivocate but not crash as introduced by FlexibleBFT [41].

In parallel to BB-Core, BlueBottle runs a secondary synchronous Byzantine Broadcast protocol, which is operated by an enlarged set of validators. We call this enlarged set (that still includes the core validators) the guard validator set. It assumes synchrony and a combined $n = 2f + 1$ fault mode. Its job is to provide enhanced

decentralization, high resilience, and a robust recovery mechanism. Our approach aims to deliver to the clients a palette of options when it comes to the latency versus security versus decentralization trade-off they wish to follow.

The core layer. Building BlueBottle’s core layer requires a high-performance consensus protocol. Our goal is simple: achieve the lowest possible finality latency and the highest experimentally demonstrated throughput. To do so, we introduce BB-Core, which builds on top the partially synchronous DAG-based consensus protocol Mysticeti [10] and reduces latency by trading fault tolerance from $n = 3f + 1$ to $n = 5f + 1$. BB-Core introduces the first $5f + 1$ DAG-based consensus protocol and commits transactions in under 0.5s while sustaining more than 200,000 tx/s (Section 5)—a performance level unmatched in the BFT consensus literature. We also introduce an asynchronous variant of BB-Core in Appendix F, which tracks the performance of BB-Core with a 33% latency increase.

We use the standard two-step design: first, consensus fixes a single order of operations for everyone, and then identical replicas apply that order ensuring they all end up in the same state (state machine replication). BB-Core exposes two explicit finality paths:

- **Fast path (2 δ):** Clients who follow every block see finality in about 1 RTT (round-trip time); under $n = 5f + 1$, this becomes roughly 0.5 RTT faster than an equivalent $n = 3f + 1$ protocol. This path is safe as long as at most f_c core validators are Byzantine.
- **Checkpoint path (4 δ):** Clients willing to wait two additional message delays finalize upon observing a checkpoint certificate (FinalityQC). After a leader slot is committed, validators propose a checkpoint over the resulting state; a critical invariant is that each honest validator proposes *at most one checkpoint per height*. This single-proposal rule guarantees that at most one valid checkpoint can exist per height, even when up to $3f_c$ core validators equivocate. Checkpoint-path clients thus remain safe if fewer than 60% of core validators are equivocating, tolerating an additional $f_{AbC} < 2f_c$ Alive-but-Corrupt [41] faults beyond the f_c Byzantine faults.

The guard layer. The guard protocol in BlueBottle is a highly decentralized auditing and recovery layer, called BB-Guard. BB-Guard operates in synchronous steps and monitors the core layer for liveness or safety violations (possible with up to $\lfloor 3n_c/5 \rfloor - 1$ faults in the core layer) in the form of equivocations. If this happens, it enables a synchronous recovery protocol. Upon detecting such misbehavior, the guard validators transition to a recovery phase. This consists of running Byzantine Broadcast of accumulated evidence and deterministically agreeing on the set of core validators that misbehaved and then excluding them, potentially by slashing their stake. If the violation relates to liveness, then the reduced set of core validators restarts consensus, whereas if it was a safety violation, they first agree on the canonical fork. We emphasize that BB-Guard is not just a simple slashing mechanism, but a full-fledged recovery protocol ensuring that the overall system can continue operating even after a significant attack on the core layer.

Contributions. We make the following contributions:

- We present BlueBottle, a novel dual-layer consensus architecture that simultaneously achieves low latency, high security, and high decentralization.
- We introduce the two main novel building blocks of BlueBottle, namely BB-Core, a DAG-based consensus protocol that reduces latency by lowering fault-tolerance to $n = 5f + 1$ and BB-Guard, a highly decentralized guard protocol for misbehavior monitoring and recovery.
- We provide a rigorous security analysis of both BB-Core and BB-Guard, demonstrating that BlueBottle achieves optimal safety and liveness guarantees under standard assumptions.
- We implement and evaluate BB-Core and compare it at scale against Mysticeti, confirming the expected latency gains of around 20–25% while maintaining similar throughput.

2 System Overview

Below we provide the system and threat models, the design goals, and a design overview of BlueBottle.

2.1 System model

We assume a total number of nodes n . BlueBottle has a *core* and a *guard* layer. All validators are *guard* validators (or guards) but only a subset is *core*. We denote the number of core validators as n_c and the number of guard validators as n . Core validators continuously operate the core consensus protocol and process transactions, while guard validators checkpoint the output and audit the operation of core validators and help with recovery. Both core and guard validators are selected using a Sybil-resistant election mechanism [26] and we usually assume a proof-of-stake approach [35]. We can translate the quantitative assumptions on the nodes to stake assumptions and we often use the two concepts interchangeably. Core validators are chosen similarly to existing quorum-based blockchains, consisting of roughly the 100 entities with the highest stake or those that meet specific criteria, such as owning a minimum percentage of the total stake [60]. Guard validators include all stakeholding entities. The number of guards is assumed to be significantly larger than the number of core validators (multiple hundreds). In practice, we expect full nodes to operate as guards. We provide a discussion on the stake distribution in the next section. BlueBottle operates as a message-passing system.

2.2 Threat model

BlueBottle assumes a computationally bounded adversary, ensuring that common cryptographic security assumptions, like those for hash functions and digital signatures, hold. We assume a computationally bounded adversary controlling at most f of $n = 2f + 1$ total validators (or equivalently, stake $S_f < S/2$ of total stake S). Byzantine nodes may behave arbitrarily; the remaining $n - f$ nodes are honest. This global bound governs the guard layer; the core layer operates under a stricter threshold ($n_c = 5f_c + 1$) that may be temporarily violated, triggering recovery. The two layers also differ in their network assumptions, detailed below. We provide a security assessment on the stake requirements for the BB-Core and BB-Guard layers at the end of this section. We also highlight that the two layers of BlueBottle conceptually operate under different networking models, which we discuss in the following paragraphs.

Table 1: Client assumptions and perceived finality latency. δ denotes the actual message delay, Δ the synchrony bound used by BB-Guard. Fast-path clients finalize upon a committed leader slot; checkpoint-path clients finalize upon a FinalityQC (Section 3.4). When the core’s fast-path threshold is exceeded, BB-Guard recovers liveness and preserves checkpoint-path safety (Section 4).

	BB-Core		BlueBottle (BB-Core + BB-Guard)		
	Fast path	Checkpoint path	Fast path	Checkpoint path	Synchronous recovery
Threshold	$f_c < \frac{n_c}{5}$	$f_c + f_{AbC} < \frac{3n_c}{5}$	$f < \frac{n}{6}$	$f + f_{AbC} < \frac{n}{2}$	$f < \frac{n}{2}$
Finality	2δ	4δ	2δ	4δ	$O(\Delta)$
Safety	✓	✓	✓	✓	Recovery preserves Checkpoints

Core layer. BlueBottle uses the novel BB-Core consensus protocol at its core, which assumes that the total number of core validators n_c satisfies $n_c \geq 5f_c + 1$, where f_c is the maximum number of Byzantine core validators which may deviate from the protocol arbitrarily. The remaining core validators are assumed to be honest and follow the protocol specification.

BB-Core exposes two client finality paths with different security-latency trade-offs, following the FlexibleBFT [41] paradigm. *Fast-path* clients finalize transactions upon observing a committed leader slot (2δ latency) and are safe as long as at most f_c core validators are Byzantine. *Checkpoint-path* clients finalize only upon observing a checkpoint certificate (4δ latency) and tolerate an additional f_{AbC} *alive-but-corrupt* (AbC) core validators, where $f_{AbC} < 2f_c$. AbC validators [41] participate in the protocol (so it does not stall) but may equivocate, i.e., sign conflicting messages. Under the checkpoint path, safety is maintained as long as the total number of equivocating core validators satisfies $f_c + f_{AbC} < 3f_c + 1$, i.e., fewer than 60% of core validators equivocate. The checkpoint mechanism is formally defined in Section 3.4 and its safety is proven in Appendix D.1. We further assume that BB-Core operates in a partially synchronous [27] network, where message delays are unbounded until a Global Stabilization Time (GST), after which they are bounded by a constant and known Δ . We also provide a variant of BB-Core for asynchronous networks in Appendix F, where message delays remain unbounded. Since the threat model of the core layer is significantly weaker than the global assumption, it can fail. However, it will only fail accountably and, as a result, allow the system to detect the adversarial nodes and exclude them, slowly converging the threat model to the operational requirements of the core layer. The process of monitoring for such accountable faults and initiating recovery is handled by the guard layer.

Guard layer. The guard layer runs BB-Guard under a synchronous network model with known delay bound Δ . Crucially, *all validators participate as guards*, including core validators; the guard set is a superset of the core set, not a disjoint group. This ensures that corrupting the guard layer requires corrupting the same validators who run the core—an adversary cannot separately target “guard-only” nodes to bypass recovery. Under synchrony and the global $n = 2f + 1$ assumption, BB-Guard detects equivocations or liveness failures in BB-Core, constructs a provable blameset of at least $f_c + 1$ misbehaving core validators, and excludes them via Byzantine Broadcast. While synchrony is a stronger assumption than partial synchrony, BlueBottle provides optimistic responsiveness: when BB-Core’s assumptions hold, clients experience fast 2δ or 4δ finality without waiting for synchronous confirmation.

Client Finality Guarantees. The client finality guarantees are a local assumption. This is on par with designs around Flexible BFT [41, 44]. Clients who believe the adversary is not actively compromising BB-Core can get the faster finality, whereas the rest can follow the checkpoint finality. Table 1 presents the fault assumptions for clients.

Stake distribution between validator groups. We show how to translate the quantitative assumptions from our threat model to stake assumptions and determine the stake distribution between core and guard validators. We assume there is a total amount of stake S available such that $S = 2S_f + 1$ where S_f is the stake controlled by the adversary. We denote the stake assigned to core validators by S_c . Given our initial stake distribution assumptions, the security assumption of the core layer might be violated at some point. However, in this case, we still want to achieve safety of checkpoints. To preserve checkpoint safety, the adversary is allowed to control no more than 60% of S_c . Let S_{c_f} denote the core stake controlled by the adversary; we must maintain the invariant

$$S_{c_f} < 3/5 \cdot S_c. \quad (1)$$

Since the adversary can concentrate its full corruption budget $S_f = (S-1)/2$ on the core validators, i.e., $S_{c_f} = (S-1)/2$, and considering that invariant 1 must nevertheless hold, we obtain

$$(S-1)/2 < 3/5 \cdot S_c \Rightarrow S_c > 5(S-1)/6. \quad (2)$$

In other words, we require that the total stake contributed by the core validators S_c is at least $\frac{5(S-1)}{6}$.

Consequently, the stake contributed by the guard validators S_g is $S - S_c < \frac{S+5}{6}$. For simplicity, we can set $S_c = S \cdot 5/6$.

2.3 Design goals

BlueBottle has the following three primary design goals in terms of performance, security, and decentralization.

- G1 High performance:** BlueBottle provides significantly lower latency and equally high throughput as state-of-the-art (DAG-based) consensus protocols when all BB-Core underlying assumptions hold.
- G2 High security:** BlueBottle can tolerate up to S_f stake being controlled by the adversary with the total amount of stake being $S = 2S_f + 1$.
- G3 High decentralization:** BlueBottle enables non-core-validator participants to contribute to the system’s security via BB-Guard.

2.4 Design overview

We present an overview of BB-Core and BB-Guard.

2.4.1 The core layer. BlueBottle introduces the novel consensus protocol BB-Core for its core layer. BB-Core builds atop an uncertified DAG [10, 30, 32], and is the first $5f + 1$ DAG-based consensus protocol. It trades off the security threshold from $n = 3f + 1$ to $n = 5f + 1$ to achieve finality in 2 message delays instead of the original 3. We prove that safety and liveness hold for BB-Core in Appendix D. The BB-Core protocol follows similar ideas to Mysticeti with the following modifications to its decision rules:

- **Direct decision rule:** Commit (skip) a leader in round R , if it has $4f_c + 1$ votes (blames) from round $R + 1$.
- **Indirect decision rule:** Commit an undecided leader in round R , if there is a committed leader in round $R + 2$ that supports the round- R leader with at least $2f_c + 1$ round- $R + 1$ votes.

2.4.2 The guard layer. BlueBottle introduces a novel guard protocol, BB-Guard, which serves three goals:

Fault detection. This is the common-case execution of BB-Guard. Guard validators synchronously gossip the blocks output by BB-Core as a commit sequence and make sure that there is no equivocated block proposal. If this is true and no equivocation is detected within the timeout, the BB-Core continues uninterrupted. Otherwise, BB-Guard has to proceed to accountability assignment.

Accountability assignment. There are two types of accountable faults in BlueBottle: safety and liveness faults. Safety faults are directly detected through the previous mechanism and, once detected, are flagged to all connected peers. Liveness faults are only detected when a core validator flags a set of $f_c + 1$ core validators as non-responsive and sends this through the gossip network for notarization. At this point, the guard validators start a timer to see if the DAG building has halted. If the timer expires, they locally decide that there are liveness faults.

Synchronous recovery. When a core validator detects a fault, it starts the synchronous recovery protocol. For this, it first decides on an accountable set of faulty validators it wishes to exclude. This set is unique and the core validator cannot change it afterwards. The accountable set comes from either safety faults (i.e., double-spending from the core), or liveness faults as described above. In either event, an honest validator (core or guard) can deduce at least an $f_c + 1$ -sized accountable set. The validator then Byzantine Broadcasts this set and all validators sign the set if they believe that the set indeed constitutes Byzantine validators. The first Byzantine Broadcast that outputs a valid set is considered the last message of the view. The validators exclude the blamed stakeholders from the new view and return to regular operation, optionally deciding on the fork they want to adopt.

3 The BB-Core Protocol

BB-Core is the first $5f + 1$ DAG-based consensus protocol. It is built on top of an uncertified DAG, similarly to Mysticeti [10] and Cordal Miners [32]. It commits in two message delays by lowering the fault threshold to $f_c < n_c/5$ ($\approx 20\%$). We state BB-Core's safety and liveness theorems inline with the protocol and defer the full proofs

Algorithm 1 BB-Core

```

leadersPerRound ▷ A number between 1 and  $4f_c + 1$ 
waveLength ▷ Set to 2 for BB-Core, 3 for BB-Core-Async

procedure TryDecide( $r_{committed}, r_{highest}$ )
   $S \leftarrow []$  ▷ Holds decisions
  for  $r \leftarrow r_{highest}$  down to  $r_{committed} + 1$  do
    for  $l \leftarrow \text{leadersPerRound} - 1$  down to 0 do
       $i \leftarrow r \bmod \text{waveLength}$ 
       $D \leftarrow \text{DECIDER}(i, l)$ 
       $w \leftarrow D.\text{WAVELENGTH}(r)$ 
       $s \leftarrow D.\text{TRYDIRECTDECIDE}(w)$ 
      if  $s = \perp$  then  $s \leftarrow D.\text{TRYINDIRECTDECIDE}(w, S)$ 
       $S \leftarrow s \parallel S$ 
  return  $S$ 

procedure ExtendCommitSeq( $r_{committed}, r_{highest}$ )
   $S \leftarrow \text{TryDecide}(r_{committed}, r_{highest})$ 
   $S_{commit} \leftarrow []$  ▷ Holds committed blocks
  for  $s \in S$  do
    if  $s = \perp$  then break
    if  $s = \text{Commit}(b_{leader})$  then  $S_{commit} \leftarrow S_{commit} \parallel b_{leader}$ 
  return  $\text{LINEARIZESUBDAGS}(S_{commit})$  ▷ Same as DAG-Rider [31]

```

to Appendix D. Algorithm 1 provides the main entry point and is invoked whenever a validator receives a new block. Algorithm 2 specifies the decision process, with each validator instantiating one *Decider* instance per leader slot. Finally, Algorithm 3 defines utility procedures that support common operations used throughout the protocol. The orange-colored lines of the algorithm refer to the asynchronous version of BB-Core which we prove in the Appendix. This is of independent interest and can be ignored. Finally, Algorithm 4 defines the checkpointing mechanisms of BB-Core.

3.1 The directed acyclic graph

BB-Core builds a *directed acyclic graph* (DAG) of blocks that reference each other via cryptographic hashes, similar to Mysticeti. This DAG enables validators to decide (locally) which blocks to commit and in what order. The BB-Core protocol proceeds in a sequence of logical *rounds*. In each round, every honest validator proposes exactly one block.

Block creation and validation. A valid block must include at least the following elements: (1) the author A of the block together with their valid cryptographic signature on the block's contents; (2) a round number R ; (3) a list of transactions; and (4) at least $4f_c + 1$ distinct hashes of valid blocks from the previous round $R - 1$.

Honest validators store only valid blocks in their local DAG and discard invalid ones. Moreover, they reference a block's hash in their proposals only if the block is valid and after downloading and verifying its entire causal history, thereby ensuring the correctness of the block's lineage. In particular, honest validators always include a reference to their own block from the previous round.

Rounds and waves. As already mentioned, BB-Core operates in *rounds* and two subsequent rounds R and $R + 1$ form a *wave*. Figure 1 (left) illustrates an example of a single wave with six validators ($v_0, v_1, v_2, v_3, v_4, v_5$). The first round R (Propose) of a wave contains the blocks that the wave attempts to commit ($P_0, P_1, P_2, P_3, P_4, P_5$) along with the equivocating block P'_1 . In the second round $R + 1$ (Decision), each block serves as a *vote* for the Propose blocks it references. In the example of Figure 1, blocks V_0, V_1, V_2, V_3 , and V_4 vote for P_0, P_1, P_2, P_3 , and P_4 (but not for P'_1 or P_5), whereas block V_5 votes for P'_1, P_2, P_3, P_4 , and P_5 (but not for P_0 or P_1). The procedure

Algorithm 2 Decider Instance

```

waveOffset = i                                ▶ The first parameter of the Decider (i)
leaderOffset = l                              ▶ The second parameter of the Decider (l)
waveLength                                     ▶ Set to 2 for BB-Core, 3 for BB-Core-Async

procedure WAVELENGTH(r)
  return (r - waveOffset) / waveLength

procedure PROPOSEROUND(w)
  return (w * waveLength) + waveOffset

procedure DECISIONROUND(w)
  return PROPOSEROUND(w) + (waveLength - 1)

procedure STRONGLYCERTIFIEDLEADER(w, bleader)
  Bdecision ← GETDECISIONBLOCKS(w)
  return |{b' ∈ Bdecision : IsVote(b', bleader)}| ≥ 4fc + 1

procedure SKIPPEDLEADER(w, bleader)
  Bdecision ← GETDECISIONBLOCKS(w)
  return |{b' ∈ Bdecision : ¬IsVote(b', bleader)}| ≥ 4fc + 1

procedure TRYDIRECTDECIDE(w)
  Bleader ← GETLEADERBLOCKS(w, leaderOffset)
  for bleader ∈ Bleader do
    if SKIPPEDLEADER(w, bleader) then return Skip
    if STRONGLYCERTIFIEDLEADER(w, bleader) then return Commit(bleader)
  return ⊥

procedure WEAKLYCERTIFIEDLEADER(banchor, bleader)
  w ← WAVELENGTH(bleader.round)
  Bdecision ← GETDECISIONBLOCKS(w)
  return |{b ∈ Bdecision : IsVote(b, bleader) ∧ LINK(b, banchor)}| ≥ 2fc + 1

procedure TRYINDIRECTDECIDE(w, S)
  rdecision ← DECISIONROUND(w)
  sanchor ← first s ∈ S s.t. rdecision < s.round ∧ s ≠ Skip
  if sanchor = Commit(banchor) then
    Bleader ← GETLEADERBLOCKS(w, leaderOffset)
    if ∃ bleader ∈ Bleader s.t. WEAKLYCERTIFIEDLEADER(banchor, bleader) then return
  return Commit(bleader)
  else return Skip
  return ⊥

```

IsVote(·) in Algorithm 3 formally defines a vote. A Propose block is considered *strongly certified* if it has at least $4f_c + 1$ votes. We say that the voting blocks form a *strong certificate* for the proposed block. In the example, blocks P_0, P_1, P_2, P_3 , and P_4 are strongly certified, while P'_1 and P_5 are not. If a block receives at least $2f_c + 1$ votes but fewer than $4f_c + 1$, it is considered *weakly certified*. As before, we also say that the voting blocks form a *weak certificate* for the proposed block. As shown in Figure 1 (right), BB-Core initiates a new wave every round: round R is a Propose round for wave 1, round $R + 1$ is a Decision round for wave 1 and a Propose round for wave 2, etc. Algorithm 2 formally defines a wave.

3.2 Proposers and anchors

BB-Core's leader slots represents a pair (validator, round) and can be either empty or contain the validator's proposal for the respective round. If the validator is Byzantine, they may have equivocated, in which case the slot would contain more than one block.

Multiple leader slots can be instantiated per round, enabling parallel leader block proposals. Each slot is in one of three states: commit, skip, or undecided. All slots begin in the undecided state, and the protocol's objective is to classify them as either commit or skip. The commit state indicates that the slot's block should be included in the total ordering, while the skip state allows the protocol to exclude slots from crashed or Byzantine validators. Crucially, the undecided state forces subsequent leader slots to

Algorithm 3 Helper Functions

```

validators                                ▶ The set of validators
async                                    ▶ Whether the protocol is asynchronous

procedure GETDECISIONBLOCKS(w)
  rdecision ← DECISIONROUND(w)
  return DAG[rdecision]

procedure GETLEADERBLOCKS(w, rank)                                ▶ Validators may equivocate
  rpropose ← PROPOSEROUND(w)
  if async then
    rdecision ← DECISIONROUND(w)
    s ← COMBINECOINSHARES(|{b.share s.t. b ∈ DAG[rdecision]})|)
  else
    s ← rpropose
  leader ← validators[(s + rank) mod |validators|]
  return {b ∈ DAG[rpropose] : b.author = leader}

procedure ISVOTE(bsupport, bleader)
  /* Note: If waveLength = 2, equivalent to LINK(bleader, bsupport) */
  function VOTEDBLOCK(b, id, r)                                ▶ Depth-first search
    if r ≥ b.round then return ⊥
    for b' ∈ b.parents do
      if (b'.author, b'.round) = (id, r) then return b'
      res ← VOTEDBLOCK(b', id, r)
      if res ≠ ⊥ then return res
    return ⊥
  (id, r) ← (bleader.author, bleader.round)
  return VOTEDBLOCK(bsupport, id, r) = bleader

procedure LINK(bold, bnew)
  return ∃ a sequence of k ∈ ℕ blocks b1, ..., bk s.t.
    b1 = bold ∧ bk = bnew ∧ ∀ j ∈ [2, k] : bj ∈ ∪r≥1 DAG[r] ∧ bj-1 ∈ bj.parents

```

Algorithm 4 Checkpoint Protocol

```

committed ← ∅                                ▶ Heights for which a commit was observed
proposed ← ∅                                ▶ Heights for which a ChkProp was broadcast
witnessed ← ∅                                ▶ Heights for which a ChkWitness was broadcast

procedure ONCOMMIT(B, s)                                ▶ Leader B committed at height s
  if s ∉ committed then
    committed ← committed ∪ {s}
    σ ← APPLY(B)
    sign and broadcast (CHKPROP, s, H(B), σ)                                ▶ Deterministic state transition

procedure ONCHKPROP((CHKPROP, s, H(B), σ))
  σ' ← APPLY(B)
  if σ' ≠ σ or H(B) malformed then return
  ChkProp ← ChkProp ∪ proposed[s]
  if |{p ∈ proposed[s] : p.σ = σ}| ≥ 4fc + 1 then
    CheckpointQC ← the 4fc + 1 matching proposals
    sign and broadcast (CHKWITNESS, s, H(B), σ, H(CheckpointQC))

procedure ONCHKWITNESS((CHKWITNESS, s, H(B), σ, H(CheckpointQC)))
  if ChkWitness malformed then return
  ChkWitness ← ChkWitness ∪ witnessed[s]
  if |witnessed[s]| ≥ 4fc + 1 then
    FinalityQC ← the 4fc + 1 matching witnesses
    return FinalityQC                                ▶ finalise height s with state σ

```

wait, preventing unsafe commitments. Similarly to related work [8, 10, 30, 53], the number of leader slots per round is a system parameter. Before advancing round, validators wait up to 2Δ time to receive the blocks from the leaders slots of the previous round, if they have not already.

3.3 The decision rule

We present the decision rule of BB-Core through an example protocol run. Figure 2 shows the local view of a BB-Core validator in a system with six validators ($v_0, v_1, v_2, v_3, v_4, v_5$), parameterized with two leader slots per round. We denote a block as $B_{(v_i, R)}$, where v_i is the issuing validator and R is the block's round. Initially, all proposer slots are in the undecided state. The validator examines

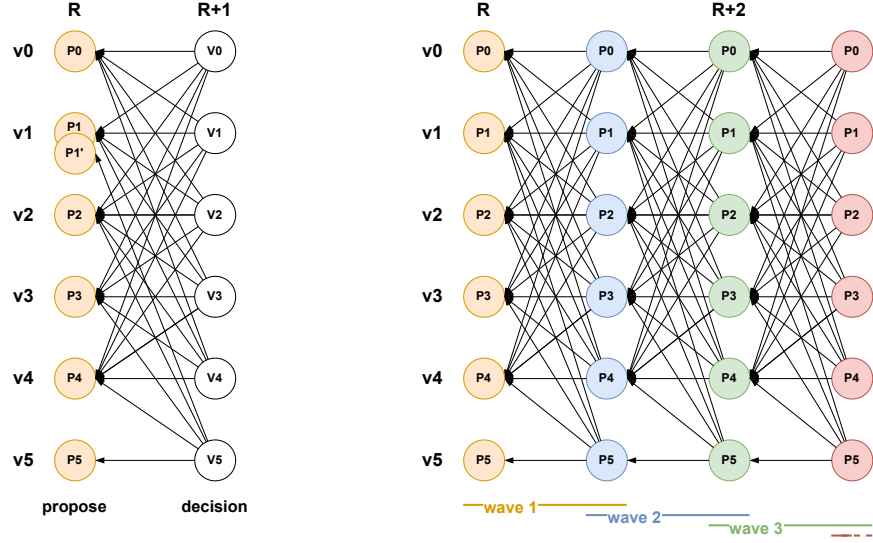


Figure 1: The structure of the BB-Core DAG. Left: A wave consisting of two rounds (Propose and Decision). Right: Wave patterns in the BB-Core protocol (each round initiates a new overlapping wave).

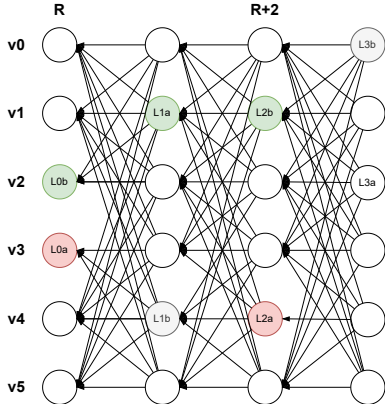


Figure 2: Example with six validators and two leader slots per round.

the portion of the DAG shown in Figure 2 and attempts to classify as many leader-slot blocks as possible into either commit or skip.

Step 1: Determine the leader slots. The sequence of leader slots for each round is predefined and known to all validators, following a deterministic round-robin rotation. In the example of Figure 2, every validator knows in advance that the leader slot sequence is $[L_{0a}, L_{0b}, L_{1a}, L_{1b}, L_{2a}, L_{2b}, L_{3a}, L_{3b}]$. This deterministic mechanism ensures that even if validators have divergent DAG views, they still agree on the leader slots (and their order) for a given round—regardless of whether a block has been observed in that slot. This design also enables low latency by allowing multiple leaders per round, using these slots to order the causal history.

Step 2: Direct decision rule. The validator attempts to classify each slot (even those without a block) as either commit or skip. To do so, it processes each slot individually, starting with the lowest slot that is part of a complete wave (L_{2b}), applying the BB-Core *direct decision rule*. The validator classifies a block B in a slot as skip if it observes $4f_c + 1$ blocks from the subsequent Decision round that do not vote for it, and as commit if it is strongly certified. As discussed in Section 3.1, a block B is strongly certified if there are $4f_c + 1$ blocks voting for it. Otherwise, the validator leaves the slot as undecided.

In this example, the validator first analyzes L_{2b} and observes that $B_{(v_0, R+3)}$, $B_{(v_1, R+3)}$, $B_{(v_2, R+3)}$, $B_{(v_3, R+3)}$, and $B_{(v_5, R+3)}$ strongly certify it. Therefore, it classifies L_{2b} as commit. Section 5 shows that this scenario is the most common (in the absence of an adversary) and results in the lowest latency. The validator then analyzes L_{2a} and observes that $B_{(v_0, R+3)}$, $B_{(v_1, R+3)}$, $B_{(v_2, R+3)}$, $B_{(v_3, R+3)}$, and $B_{(v_5, R+3)}$ do not vote for it. Therefore, it classifies L_{2a} as skip. The presence of $4f_c + 1$ blocks from the Decision round that do not vote for a block ensures that it will never be certified, and will thus never be committed by other validators with a potentially different local view of the DAG. Section 5 shows that this rule allows BB-Core to promptly skip (benign) crashed leaders to minimize their impact on the protocol's performance.

Step 3: Indirect decision rule. In the case where the direct decision rule cannot classify a slot, the validator uses the BB-Core *indirect decision rule*. This rule looks at future slots to decide about the current one. First, it finds an *anchor*. This is the earliest leader slot with a round number $R' > R + 1$ that is either still classified as undecided or already classified as commit. If the anchor is undecided, the validator marks the current slot as undecided. If the anchor is commit, the validator checks if it references at least one weak certificate over the current slot. If it does, the validator

marks the current slot as `commit`. If it does not, the validator marks the current slot as `skip`. We show in Appendix D that the direct and indirect decision rules are consistent, namely if one validator direct commits a block no honest validators will indirect skip it and vice versa.

In this example, the validator fails to classify L_{0b} using the direct decision rule as it is neither strongly certified nor are there enough blocks from the subsequent Decision round that do not reference it to classify it as `skip`. It thus searches for its anchor. Since L_{2a} has been classified as `skip`, it cannot serve as an anchor; therefore, L_{2b} is the anchor for L_{0b} . Given that L_{2b} references $B_{(v_0, R+1)}$, $B_{(v_1, R+1)}$, and $B_{(v_2, R+1)}$, which form a weak certificate for L_{0b} , the validator classifies L_{0b} as `commit`. Finally, L_{0a} cannot be classified using the direct decision rule as well. Its anchor is again L_{2b} , but it does not reference a weak certificate over L_{0a} . Therefore, the validator classifies L_{0a} as `skip`.

Step 4: Commit sequence. After processing all slots, the validator derives an ordered sequence of the leader-slot blocks. It then iterates over this sequence, committing all slots marked as `commit` and skipping all slots marked as `skip`, until it encounters the first undecided slot. As shown in Appendix D, this commit sequence is safe, and eventually, every slot is classified as either `commit` or `skip`. In the example of Figure 2, the leader sequence is $[L_{0b}, L_{1a}]$.

Following the approach introduced by DagRider [31], the validator derives the final commit sequence by linearizing the blocks within the sub-DAG defined by each leader block using a depth-first search. If a block has already been included by a previous leader slot, it is not re-linearized. Leader slots are processed sequentially, ensuring that all blocks appear in the final commit sequence exactly once and in an order consistent with their causal dependencies. The procedure `LINEARIZESUBDAGS( $\cdot$ )` (Algorithm 1) formalizes this step. In the running example, the commit sequence is $[L_{0b}, B_{(v_0, R)}, B_{(v_1, R)}, B_{(v_4, R)}, B_{(v_5, R)}, L_{1a}]$.

3.4 Client finality paths and checkpoints

BB-Core exposes two finality paths to clients, each offering a different trade-off between latency and fault tolerance. Both paths use the same quorum threshold of $4f_c + 1$. Algorithm 4 formalizes the checkpoint protocol.

Fast path (2 δ finality). A fast-path client finalizes a transaction as soon as the leader slot containing it is committed via the direct or indirect decision rule described above. This requires a single wave (two message delays) in the common case when the leader is directly committed. Fast-path safety holds as long as at most f_c of the $n_c = 5f_c + 1$ core validators are Byzantine (Theorem 1); liveness after GST is established by Theorem 3 and agreement by Theorem 4 (see Appendix D).

Checkpoint path (4 δ finality). Checkpoint-path clients trade latency for stronger fault tolerance by waiting for a *checkpoint certificate*. The checkpoint mechanism chains two additional certificate steps on top of the commit path, as follows.

Step 5: Checkpoint proposal. After a leader slot at height s is committed (i.e., a strong certificate exists for the leader block B at round R), each validator computes the deterministic state transition $\sigma = \text{APPLY}(B)$ and then broadcasts a *checkpoint proposal* $\langle \text{CHKPROP}, s, H(B), \sigma \rangle$. Crucially, **an honest validator broadcasts**

at most one checkpoint proposal per slot height, even if it later observes a competing committed branch. This single-proposal rule is the key invariant that prevents conflicting checkpoints. When $4f_c + 1$ matching checkpoint proposals for the same $(s, H(B), \sigma)$ are collected, they form a *CheckpointQC*.

Step 6: Checkpoint witness. If a validator observes a valid CheckpointQC for height s , it broadcasts a *checkpoint witness* $\langle \text{CHKWITNESS}, s, H(B), \sigma, H(\text{CheckpointQC}) \rangle$. An honest validator broadcasts a witness as long as the checkpoint’s state σ matches its own deterministic execution of the payload, even if its single permitted checkpoint proposal for that height was already spent on a different, unfinalized branch. When $4f_c + 1$ matching witnesses are collected, they form a *FinalityQC*. A checkpoint-path client finalizes height s upon observing the FinalityQC together with its underlying CheckpointQC.

Latency derivation. The four message delays of this path are:

- (1) The leader proposes a block (round R).
- (2) Validators vote; $4f_c + 1$ votes yield a strong certificate, committing the leader (round $R + 1$). *Fast-path clients finalize here.*
- (3) Validators broadcast checkpoint proposals; $4f_c + 1$ matching proposals form a CheckpointQC (round $R + 2$).
- (4) Validators broadcast checkpoint witnesses; $4f_c + 1$ matching witnesses form a FinalityQC (round $R + 3$). *Checkpoint-path clients finalize here.*

In practice, checkpoint proposals and witnesses are embedded as metadata in subsequent DAG blocks (rounds $R + 2$ and $R + 3$) rather than sent as separate messages, keeping the mechanism integrated into the DAG structure.

Checkpoint-path fault tolerance. The checkpoint path tolerates an additional $f_{AbC} < 2f_c$ alive-but-corrupt (AbC) validators [41]—validators that participate in the protocol but may equivocate. Under this extended threat model, the total number of equivocating validators can reach up to $f_c + f_{AbC} < 3f_c + 1$, i.e., fewer than 60% of the $5f_c + 1$ core validators. Safety is guaranteed because the single-proposal rule ensures that at most one CheckpointQC can exist per height (Lemma 17, Appendix D), even when up to $3f_c$ validators equivocate; the end-to-end checkpoint-path safety statement is Theorem 5. If the fast-path safety threshold is exceeded (i.e., more than f_c validators equivocate) and a fork occurs at the commit level, the checkpoint path remains safe: honest replicas halt and rely on BB-Guard’s recovery mechanism (Section 4) to resolve the fork, using the unique CheckpointQC chain as the canonical history.

4 The BB-Guard protocol

While the previous section focuses on how BB-Core achieves low latency, this holds under the assumption of fewer corruptions and medium-level of decentralization, which introduces the practical risk of a weaker resilience threshold. We now present an approach to further mitigate any security concerns, by introducing an additional (slow) layer of higher resilience and decentralization. That is BB-Guard, a protocol that monitors the operation of BB-Core in a synchronous pace, and corrects potential violations via two guarantees; (i) any safety or liveness violation will be caught and (ii) at least $f_c + 1$ violating core validators will be provably identified

by all participating validators. In practice, BB-Guard will operate in a synchronous, slower pace than BB-Core, while verifying that the main protocol is operating correctly, and recovering liveness and safety in case any of the two fails due to more than f_c corruptions. All BB-Core validators are also BB-Guard validators. We present the main BB-Guard functionality in Algorithm 5 and some helper functions in Algorithm 6. While BB-Guard is specifically designed for BB-Core, its framework can be generalized to be deployed along different consensus protocols.

4.1 Motivation

The BB-Guard protocol ensures safety and liveness of BlueBottle when the system is under attack. Clients can opt to utilize BB-Guard and exchange latency to gain in resilience in scenarios with higher decentralization. We argue via two constructive examples.

Example 1: Liveness failure. Assume that during the operation of BB-Core, the resilience assumption breaks, and an adversary $\mathcal{A}$ takes control of $f_c + 1$ validators. In such a case, it is simple to see that our protocol loses liveness indefinitely, even via crash-faults only. The protocol requires $4f_c + 1$ votes for each round's slots to be determined and for it to move forward on the next round. With fewer than $4f_c + 1$ honest validators participating, the protocol could be skipping rounds or make no progress indefinitely.

Example 2: Safety failure. Again, assume that during the operation of BB-Core, the resilience assumption breaks, and an adversary $\mathcal{A}$ takes control of $3f_c$ core validators. The adversary is then able to present to some validators a directly committed leader block B_L ($4f_c + 1$ support), whereas for others it will be undecided ($f_c + 1$ support). The next leader will also be presented the latter case, so in its view B_{L+1} will not have enough support to indirectly commit B_L . When B_{L+1} is directly committed, the validators will diverge views (some will already have B_L committed whereas others will skip it) and safety will break.

Both examples showcase the importance of maintaining the resilience assumption for BB-Core to operate correctly. However, in practice, it is possible that an adversary could temporarily break the resilience assumption. In such a case, we would like to ensure that our protocol can recover both safety and liveness, which is our main goal with BB-Guard.

4.2 BB-Guard setting

Before diving into the protocol details, we recall the overall setting: We assume a set of n validators who in total hold stake S such that each validator maintains stake equivalent to S/n ¹. We assume that there exists a polynomially bounded adversary $\mathcal{A}$ who can corrupt up to a total of $S_f \leq (S - 1)/2$ stake in a static fashion, i.e. $\mathcal{A}$ picks which validators to corrupt (up to the corruption threshold) before the protocol starts, and cannot change corruptions after.

For this part of our construction, we assume that validators communicate over a synchronous, point-to-point protocol, with a known network delay Δ_{net} . We also denote by Δ , the *network delay*

for *reliable broadcast*. This is the theoretical delay with which we will analyze our guard protocol².

This is also the Δ timeout used for the liveness proofs of BB-Core, however, in the practical deployment we optimistically wait a much smaller timeout before blaming a leader. This practice has been introduced by Shoal [56], where the real Δ timeout is only used when many consecutive leaders are skipped. It balances well the practical needs with the theoretical requirements and is what most production systems follow.

4.3 Monitoring failures

Block handling. Upon receiving a block, each guard verifies that the block is valid with respect to BB-Core, by executing the pre-determined validity checks defined by the BB-Core protocol. Furthermore, guards check for equivocations with respect to every received block. They do so, by maintaining state of what blocks they have received from which core validators for each round of BB-Core. If a block is equivocated by its authoring validator, this is proof of misbehavior for that party. In the current construction, guards simply do not consider any such blocks at all. However, more sophisticated designs could allow such blocks to contribute to the provable equivocations of the system, so that malicious parties are identified via such equivocations as well.

Blamesets. Blamesets are the main gadget of our construction. A *valid blameset* is a set of at least $f_c + 1$ core validators, that have provably misbehaved. There are two types of blamesets, namely *liveness* and *safety* blamesets, according to the type of misbehavior the validator has been proven to have performed. Each valid blameset is accompanied by a valid proof of misbehavior for each validator in it. Once an honest guard has established a new valid blameset, it can request a recovery process, via which, guards will agree on one blameset, and slash the validators in that blameset.

Liveness failure detection. Guards are also tasked with restoring the core protocol's liveness. To check for potential liveness failures, for each new round, they maintain a timer Δ_{live} and a set asleep that contains inactive core validators and is initialized to core, the set of all core validators. Upon receiving a valid block from some core validator v_i for round r , the guard will remove v_i from asleep(r), indicating that the validator is active for the round. After the timer expires, any validator remaining in asleep, will be blamed by the guard. After the blaming phase, validators have a chance to respond within a grace period Δ_{grace} . During the grace period, blamed (or not) validators can respond by providing a valid block for the round produced by the the blamed validator, to convince all others that the blamed party is live. After the grace period, any validator who is not convinced, votes against the blamed party. If enough (majority) votes are gathered, validators can use them as proof for the inclusion of the party in their liveness blameset.

Safety failure detection. Honest validators are guaranteed to not equivocate, and to not vote for equivocating blocks. However, if the safety assumption of the BB-Core protocol breaks, i.e., the adversary can corrupt more than f_c validators, then equivocating blocks can be committed (still, no two equivocating blocks can both be directly

¹This model can be easily simulated even in a setting where validators hold unequal units of stake, by having each validator simulate a separate Sybil validator per each unit of stake it holds.

²In practice, we will set Δ to a sufficiently large time so that all messages are guaranteed to be delivered by then. Since this construction is going to be part of the slow path, we can set Δ to a large number.

committed, as long as the corruptions remain $\leq 3f_c$). However, as we described before there could be an equivocation through the indirect skip path. In this case, BB-Guard will come in play; every time a guard observes a new set of blocks that are being committed to the DAG, they compare existing already committed, leader blocks to the new ones. In case a guard finds two equivocating blocks, it compares their support, to find the set of overlapping parties supporting the two. This overlapping set, which as we show will be provable and will contain at least $f_c + 1$ misbehaving parties, can then be used for recovery/slashing. Equivocation is meant in a broad sense and while we focus on BB-Core this generalizes to any consensus protocol where equivocating blocks or votes can exist.

4.4 Recovery

In this construction, we show how guards can catch liveness/safety issues, and in such cases, agree on a blameset of at least $f_c + 1$ Byzantine parties. Honest guards can agree simply by running Byzantine agreement on the recovery set of each guard, which they can do since they maintain honest majority. After that, they can deterministically choose the first (in order) valid blameset, as the set of core validators to be removed.

After honest guards agree on such a blameset, they can provably notify the core validators. They in turn can then disregard the participants proven to be malicious (or in practice, slash their stake, which leads to future research towards cryptoeconomic/incentive-based security of such systems). The core validators can, at that point, execute an honest-majority agreement protocol, since they will have regained majority (by removing $f_c + 1$ malicious out of $5f_c + 1$ total, the new split is $2f_c + 1$ honest out of $4f_c$ total). For this reason, we focus on showing how to regain honest majority in the core validators in cases of misbehaviour, and not on the specifics of how to utilize the honest majority to recover the protocol afterwards.

4.5 BB-Guard Safety and Liveness

LEMMA 1. *Assume there are $n_c = 5f_c + 1$ core authorities and up to $3f_c$ of them are malicious. No two honest authorities can directly commit different leader blocks (or directly commit and directly skip) for the same leader and round.*

PROOF. Assume that this can occur. Then, there are $4f_c + 1$ authorities which vote $B_{(l,r)}$ and $4f_c + 1$ authorities which vote (or skip) $B'_{(l,r)}$. Since there are $3f_c$ malicious authorities, there are still $f_c + 1$ honest votes needed for $B_{(l,r)}$ and another $f_c + 1$ honest votes needed for $B'_{(l,r)}$ (or to skip B'). This means that an honest authority must do both, a contradiction. $\square$

COROLLARY 1. *The safety guarantee of Lemma 9 holds even under $3f_c$ corruptions.*

PROOF. Same argument as in the respective proof, but with $5f_c + 2$ authorities required, instead of $7f_c + 2$. This is an instance of the same $2q - n_c = 3f_c + 1$ quorum-intersection argument used in Lemma 17: any two $q = 4f_c + 1$ quorums over $n_c = 5f_c + 1$ validators intersect in at least $3f_c + 1$ parties, so forming two conflicting quorums requires at least $3f_c + 1$ equivocators. $\square$

LEMMA 2. *If there exists a block which an honest core authority directly commits while another honest core authority indirectly*

Algorithm 5 BB-Guard Main Functions

```

procedure ONROUND( $r$ )
  Set  $now = r - 1$ 
  Set timers  $\Delta_{live} = 4\Delta$ ,  $\Delta_{leader} = 2\Delta$ 
  if  $\Delta_{leader}$  fires then
    Set  $now = r$ 
    for  $l \in \text{GETLEADERS}(r - 1) : l \in \text{asleep}(r - 1)$  do
      BC(LBlame,  $i, l, r - 1$ )
  if  $\Delta_{live}$  fires then
    for  $core_j \in \text{asleep}(r)$  do
      BC(LBlame,  $i, core_j, r$ )
    for  $l \in \text{GETLEADERS}(r - 1) : l \notin \text{asleep}(r - 1)$  do
      for  $core_j : \neg \text{IsVote}(b_{(j,r)}, b_{(l,r-1)})$  do
        BC(LBlame,  $i, core_j, r$ )
  Set timer  $\Delta_{grace} = 2\Delta$ 
  if  $\Delta_{grace}$  fires and  $|\text{LBlamed}(r)| \geq f_c + 1$  then
    RECOVER( $i, \text{LBlamed}(r), \pi_r$ )
  procedure ONBLOCK( $b := B_{(u,r)}$ )
    if Valid( $b$ )  $\wedge$  Equivocates( $b$ ) =  $\perp$   $\wedge$   $now \leq r$  then
      asleep( $r$ )  $\leftarrow$  asleep( $r$ )  $\setminus u$ 
      BC( $(i, b, \sigma_i(b))$ )
  procedure ONCOREUPDATE( $S$ )
    ( $Set, \pi$ )  $\leftarrow$  CHECKEQUIVOCATION( $S$ )
    if  $Set \neq \perp$  then
      RECOVER( $(i, Set, \pi)$ )
    else
      for  $b \in S$  do
        BC( $(i, b, \sigma_i(b))$ )
  procedure ONLBLAME( $(j, core_k, r)$ )
    blames( $k, r$ )  $\leftarrow$  blames( $k, r$ )  $\cup \{j\}$ 
    if Maj(blames( $k, r$ )) then
      LBlamed( $r$ )  $\leftarrow$  LBlamed( $r$ )  $\cup \{k\}$ ,  $\pi_r \leftarrow \pi_r \parallel \text{blames}(k, r)$ 
  procedure ONRECOVER( $(j, S_B, \pi)$ )
    if isValidBlameSet( $S_B, \pi$ ) and  $S_i = \perp$  then
       $S_i \leftarrow S_B$ 
      RECOVER( $i, S_i, \pi$ )

```

Algorithm 6 BB-Guard Helper Functions

```

validators ▷ The set of validators

procedure GETLEADERS( $r$ )
  Let  $l = \text{leadersPerRound}$ ,  $V = |\text{validators}|$ 
  return  $\{\text{validators}[(r + d) \bmod V] : d = 0, \dots, l - 1\}$ 
procedure RESOLVEEQUIVOCATION( $b, b'$ )
   $Set \leftarrow \{v \in \text{validators} : \text{supporting } b \text{ and } b'\}$ 
   $\pi \leftarrow \{\text{respective blocks showing support from } Set\}$ 
  return ( $Set, \pi$ )
procedure CHECKEQUIVOCATION( $S$ )
  if  $\exists b \in S : B \leftarrow \text{EQUIVOCATES}(b, \text{LocState}()) \wedge B \neq \perp$  then
    ( $Set, \pi$ )  $\leftarrow$  RESOLVEEQUIVOCATION( $B, b$ )
    return ( $Set, \pi$ )
  return  $\perp$ 
procedure RECOVER( $j, S_B, \pi$ )
  BC( $i, S, \sigma_i(S, \text{recover})$ )
  Set RCVec $_i = \langle S_j \rangle_{j \in [n]}$ , where  $S_j$  is:
    
$$\begin{cases} S_j, & \text{if received unique: } (j, S_j, \sigma_j(S_j, \text{recover})) \text{ within } \Delta \\ 1, & \text{else.} \end{cases}$$

  BA( $j, S_j$ )  $\rightarrow$   $SR_j$ , for all  $j \in [n]$ 
  ORDER( $\{SR_j\}_{j \in [n]}$ )  $\rightarrow$   $\mathbf{O}$ 
  return the first valid  $SR_j \in \mathbf{O}$ 

```

skips, then there exist at least $f_c + 1$ authorities that have provably equivocated.

PROOF. Assume that such a block exists and call it B . Each block links to $4f_c + 1$ blocks from the previous round. Assume that a block B' in round $R' > R$ does not link to $2f_c + 1$ support for B from round R (otherwise, from Lemma 11, Corollary 3 would apply and B would never be indirectly skipped). In that case, focus on $R' = R + 1$. B' links to $4f_c + 1$ blocks from round R . Since $4f_c + 1$ blocks in round R support B , and the total of distinct core authorities is $5f_c + 1$, the

minimum number of equivocations is $f_c + 1$, and it is provable by their difference in votes. $\square$

LEMMA 3. *If there exists a block that an honest core authority directly skips while another honest core authority indirectly commits, then there exist at least $f_c + 1$ authorities that have provably equivocated.*

PROOF. Assume that such a block exists and call it B . Thus, there are $4f_c + 1$ authorities which blame B and $2f_c + 1$ authorities which support B . This requires a total of $6f_c + 2$ votes, out of which only $5f_c + 1$ can be distinct. As a result $6f_c + 2 - (5f_c + 1) = f_c + 1$ need to both support and blame B constituting a proof of equivocation. $\square$

LEMMA 4. *If there exist two honest core authorities who have decided on a leader block and do not agree on the decision, then there exist at least $f_c + 1$ core authorities that have provably equivocated.*

PROOF. Let $B_X \neq B_Y$ be the committed (or skipped) leader blocks according to authorities X and Y respectively for leader l in round r . By Lemma 1, no two honest authorities could have both directly committed conflicting blocks. Similarly, no two honest authorities could have directly committed and directly skipped a block respectively (Corollary 1). The only viable cases are the following:

Case: X directly committed B_X , while Y directly skipped B_Y (and vice versa): From Lemma 2 (and Lemma 3 respectively), there will exist a set of at least $f_c + 1$ provably misbehaving parties.

Case: X directly committed B_X , while Y indirectly committed B_Y (and vice versa): Similar to previous arguments, this requires at least $(4f_c + 1) + (2f_c + 1)$ distinct votes, i.e., at least $f_c + 1$ provable equivocations.

Case: X indirectly committed B_X , while Y indirectly committed B_Y : Let A^X and A^Y be the directly-committed anchors used by X and Y to indirectly commit B_X and B_Y , respectively. Since no honest party could have voted for both anchors, there are not enough votes for both anchors to be directly committed (total votes needed $8f_c + 2$, but total votes on the protocol are $2f_c + 1 + 6f_c$).

Case: X indirectly committed B_X , while Y directly skipped B_Y : In that case, let A denote the directly-committed anchor used to indirectly commit B_X , and A' denote the directly-committed anchor used to indirectly skip B_Y . Again, since no honest party could have voted for both anchors, there are not enough votes for both anchors to be directly committed. $\square$

LEMMA 5. *No honest core validators will ever be included in a valid safety-blameset.*

PROOF. Honest validators receive blocks and vote based on whether the blocks preserve the rules of the protocol. The validators' votes cannot be equivocated, and honest validators will never vote for two conflicting blocks. As such, honest validators can not be included in any valid blameset. $\square$

LEMMA 6. *No honest core validators will ever be included in a valid liveness-blameset.*

PROOF. Assume that an honest core validator v is correctly included in a liveness-blame for some round r , i.e., at least $S_f + 1$

guards have attested that they blamed v for round r . At least one of the guards is honest, say g . Then, according to Algorithm 5, either i) v did not vote for a leader that g considers live, or ii) v did not vote on time, or iii) v was a leader and did not send on time.

For the first case, g considers leaders on time, only if it receives a valid leader block for the previous round within 2Δ after the round has started. Even if the previous round progressed instantly, within Δ , v must have also progressed and received the leader's block. If v then voted for the leader, within another Δ , v 's block voting for the leader would arrive to g , on time (within the 4Δ duration of Δ_{live}^g). As such, v must have seen a leader block on time, and still not voted for it, which contradicts the honest behavior.

Similarly for the second case, g considers on time any valid vote that arrives within 4Δ after g enters the round. Since v will enter the round by at most a Δ delay after g , if v were to follow the protocol, it would wait by at most another 2Δ for the leader's block and then submit its vote. Since it takes at most an additional Δ for v 's vote to arrive at g , v must have entered the round on time, and still not voted for it on time, which contradicts the honest behavior.

Finally, if v was a leader for a round, it enters the round within at most Δ after the first honest party. An honest leader proposes a block upon entering a round for which it is the leader. Since g would wait for 2Δ for the leader's block after entering the round, even if v entered the round Δ time after g , it would still be on time to send its block to g . Thus, v must have entered the round on time, but not have sent its block on time, which contradicts honest behavior. $\square$

COROLLARY 2. *If the number of corruptions is $\leq f_c$, then no Recover call occurs.*

PROOF. Implied by Lemmas 5, 6, since there are not enough misbehaving parties to construct blamesets. $\square$

LEMMA 7 (SAFETY VIOLATIONS). *Any safety violation (equivocation) on BB-Core will be identified, and restored within $2\Delta + \Delta_{BA}$ of the moment the first honest party observes it.*

PROOF. Assume that an honest party (validator or guard) observes two equivocating blocks (according to its view) at time t . Due to synchrony, after Δ , all honest parties will be observing a superset of the validator's view, and can also see the equivocation. Such equivocations can always construct an $f_c + 1$ blameset (per Lemma 4) and as such by time $t + 2\Delta$, all honest guards will be running BA , some with input the blameset of the aforementioned equivocation. Within another Δ_{BA} , all guards will agree on a blameset of at least $f_c + 1$ validators, (since there exists at least one such valid blameset), and will recover the safety of the protocol. $\square$

LEMMA 8 (LIVENESS). *For every round r , BB-Core progresses from r within 6Δ of entering r , or at least $f_c + 1$ parties are blamed in BB-Guard.*

PROOF. First, parties enter round r of BB-Core, once they receive $4f_c + 1$ blocks (from distinct senders) from round $r - 1$. Every guard will also enter rounds according to this logic. From synchrony and broadcast, it is guaranteed that once a guard enters round r , every participant will enter round r at most within Δ as well (similar to Lemma 13). Each guard sets a timer $\Delta_{\text{live}} = 4\Delta$, by which it expects

to receive blocks from at least $4f_c + 1$ core validators; this is because in the worst-case, it would take at most an additional Δ for the slowest validator to enter the round, it would wait for the leaders of the past round for 2Δ and would take an additional Δ for its block to return to the guard. Each guard expects to have received blocks from the leaders of the past round already before the current round starts, or within 2Δ of the current round start; this is because in the worst-case, it would take an additional Δ for the slowest leader to enter the past round, and it would take another Δ for its block to arrive at the guard.

Say g^* is the first honest guard that enters round r , and let $\text{asleep}_{g^*}(r)$ denote the set once $\Delta_{\text{live}}^{g^*} = 4\Delta$ expires. Within another Δ after $\Delta_{\text{live}}^{g^*}$ (5Δ total), every other guard's (say g) timer will also expire and they will be blaming each of the asleep core validators $v \in \text{asleep}_g(r)$. If $|\text{asleep}_g(r)| \leq f_c$ for any $g \in H$, with H denoting the set of all honest guards, then it is guaranteed that all guards can progress the round after another Δ (6Δ total). The same holds if $|\cap_{g \in H} \text{asleep}_g(r)| \leq f_c$, since locally, every guard by that time will have blocks from all $v \notin \cap_{g \in H} \text{asleep}_g(r)$, which are sufficient to progress the BB-Core round. Otherwise, by 6Δ total, all guards will have at least $|H| = S_f + 1$ blames for all $v \in \cap_{g \in H} \text{asleep}_g(r)$, where $|\cap_{g \in H} \text{asleep}_g(r)| \geq f_c + 1$. $\square$

5 Evaluation

We implement a BB-Core validator³ in Rust by forking the Mysticeti codebase [38]. We provide more details about our implementation and testing methodology in Appendix E.1. We evaluate the throughput and latency of BB-Core through experiments conducted on Amazon Web Services (AWS) on a geo-distributed testbed. We detail the experimental setup in Appendix E.2.

Evaluation scope. Our evaluation has one key goal: demonstrate that fault tolerance can be traded for lower latency. We do not claim that BB-Core is the best or most performant protocol in the $n = 5f + 1$ design space. As a result, our primary baseline is Mysticeti [10], one of the closest $n = 3f + 1$ consensus protocols to BB-Core in terms of design and implementation.

Specifically, we demonstrate the following claims:

- C1** BB-Core has similar throughput and lower latency than Mysticeti when operating in fault-free, synchronous networks.
- C2** BB-Core scales as well as Mysticeti and maintains high throughput and low latency as the number of validators increases.
- C3** BB-Core has a similar throughput to and lower latency than Mysticeti in the presence of (benign) crash faults.

Note that evaluating the performance of BFT protocols in the presence of Byzantine faults is an open research question [12], and state-of-the-art evidence relies on formal proofs (presented in Appendix D).

Baseline protocols. Recent work has proposed several consensus protocols that explore similar designs to the $n = 5f + 1$ setting (Section 7), including Minimmit [21], Kudzu [50], Alpenglow [33].

None of these protocols provide deployable implementations with networking code; their codebases are intended for simulation only.

Hydrangea [52] operates with $n = 3f + 2c + k + 1$, where c and k are additional parameters that allow trading fault tolerance for latency. Hydrangea can commit in a single round-trip and is, to our knowledge, the only protocol in this design space with a deployable implementation; we therefore include it as a secondary baseline. Following the authors' specifications [52], we set $f = 1$, $c = 2$, $k = 2$ for experiments with 10 validators and $f = 9$, $c = 10$, $k = 2$ for experiments with 50 validators.

5.1 Benchmark under ideal conditions

We evaluate the performance of BB-Core under normal, failure-free conditions in a wide-area network (WAN). Figure 3 (Left) reports results from a geo-replicated deployment with a small committee of 10 validators and a larger committee of 50 validators. For cost reasons, we cap the input load at 100,000 tx/s (10 validators) and 300,000 tx/s (50 validators). These rates are two orders of magnitude above the observed peak throughput of any existing blockchain and we believe sufficient to assess system behavior under load.

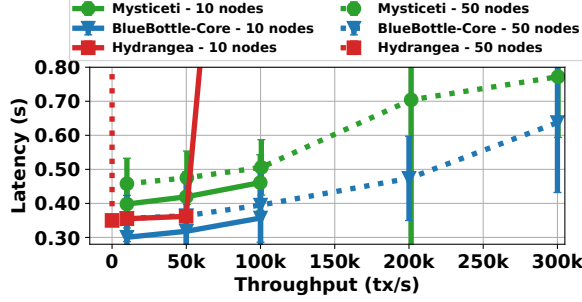
The figure shows that BB-Core successfully trades some fault tolerance for lower latency. BB-Core lowers the commit path to two message delays. Each round of BB-Core must, however, wait for a larger parent quorum (80% of validators) than Mysticeti (67%). Despite this larger quorum, BB-Core reduces end-to-end latency by about 20–25% in the WAN setting across all input loads, confirming claim **C1**. Concretely, with 10 validators at 100,000 tx/s, BB-Core attains about 357 ms latency versus about 461 ms for Mysticeti; with 50 validators at 100,000 tx/s, BB-Core reaches about 395 ms versus about 505 ms, and at 300,000 tx/s, BB-Core reaches about 637 ms versus about 772 ms. We observe a consistent proportional latency reduction at both committee sizes, confirming claim **C2**.

Hydrangea's performance is significantly worse than both Mysticeti and BB-Core (red lines in Figure 3). With 10 validators under no faults, Hydrangea sustains competitive latency (~355 ms) up to around 50,000 tx/s but degrades sharply at higher loads. With 50 validators or under crash faults, Hydrangea's latency exceeds the visible scale of the plot even at minimal offered load, indicating that the protocol does not scale gracefully to larger committees in its current form. Hydrangea is a fork of HotStuff [55] that embeds data directly in the leader's proposal; its limited scalability is thus likely attributable to the data-dissemination mechanism inherited from HotStuff, consistent with the findings of Danezis et al. [23].

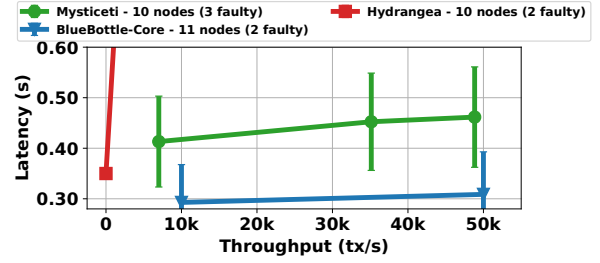
5.2 Benchmark under faults

Figure 3 (Right) shows BB-Core and Mysticeti under 3 crash faults in committees of 10 and 11 validators, (the minimum sizes used for this fault level in our experiments). We cap the offered load at 50,000 tx/s for cost reasons. As expected, both systems sustain the input load with no material latency inflation relative to the fault-free runs. Mysticeti's latency lies between 400 and 500 ms, whereas BB-Core remains between 300 and 350 ms, preserving a 20–25% latency advantage (again at the cost of lowering fault tolerance) and confirming claim **C3**. Both systems handle benign crashes gracefully by skipping faulty leaders via the direct skip rule described in Section 3. Hydrangea's latency again exceeds the

³ <https://github.com/phvv/mysticeti/tree/odontoceti> (commit e02aeba)



(a) Committees with 10 and 50 validators, no validator faults.



(b) With 3 or 2 crash faults; committees with 10 and 11 validators (minimum to tolerate 3 or 2 faults).

Figure 3: WAN throughput-latency performance comparison of BB-Core ($n = 5f + 1$) and Mysticeti ($n = 3f + 1$) with a 512B transaction size. The y-axis starts at 300 ms to zoom in on the latency difference between the systems.

visible scale of the plot, confirming its limited ability to handle faults gracefully, inline with the findings of Danezis et al. [23].

6 Making the Core Asynchronous

We study BB-Core under partial synchrony and also introduce an asynchronous variant. The asynchronous algorithm mirrors the partially synchronous one, except for the orange-highlighted additions, with formal correctness proofs in Appendix F. Using the asynchronous BB-Core in BlueBottle introduces a subtle effect: BB-Guard must detect liveness faults via a different mechanism.

In the partially synchronous design, liveness monitoring first checks whether the round leader is faulty and then verifies that the received blocks correctly reject a faulty leader. This procedure uses a timer of 3Δ , where Δ denotes the network delay bound. The asynchronous variant removes leader timeouts, so liveness detection completes in 2Δ . This yields two implications. The first one is straightforward: the partially synchronous protocol trades one round of optimistic-case latency for faster liveness recovery in the asynchronous protocol. The second one is more significant: the asynchronous variant removes the dependence on the network delay Δ between BB-Core and BB-Guard. This decoupling lets operators choose a conservative Δ (even tens of seconds) without harming BB-Core’s performance under crash faults. Operators can adjust this value at every epoch. Moreover, a faulty-leader detection and exclusion mechanism, such as HammerHead [61], further mitigates the impact of faulty leaders.

7 Related Work

Byzantine fault-tolerant state machine replication (BFT-SMR) is a foundational abstraction in distributed systems, and its latency limits have been studied extensively [4, 5, 19, 29, 34, 42]. Classical work emphasizes worst-case latency, with state-of-the-art protocols committing in three communication steps while tolerating up to $f < n/3$ Byzantine faults [16, 19, 20].

Motivated by practical deployments (e.g., blockchains), recent work increasingly targets good-case latency under partial synchrony, i.e., the latency to commit when the designated leader is correct and the network satisfies partial synchrony assumptions [6, 37]. In this setting, protocols aspire to two-round commitment. SBFT [29] achieves two rounds in the absence of Byzantine faults and reverts to a slower path otherwise. FAB [42] removes this restriction but was later shown to suffer from a liveness issue [1].

Kudzu [50] commits in three rounds in general and in two rounds when the number of Byzantine faults is small ($f < n/5$). Minimit [21] is a recently proposed 2-round variant of Simplex [7, 20, 49] with $n = 5f + 1$, and reports, similarly to Kudzu, latency results only through simulation based on the protocol’s finalization rule, i.e., the number of communication steps and threshold required before a block can be committed. Minimit and Kudzu’s fast path use the same finalization structure, requiring one round of communication with a $4f + 1$ threshold, and therefore produce identical block latency (see Tables 3, 5, 6, 7 of [21]). Hydrangea [52] adopts a generalized fault model that distinguishes Byzantine from crash faults; while this model is weaker than a purely Byzantine model for a fixed total fault budget, it requires careful parameterization for practical deployment. Alpenglow [33] claims two-round commitment while simultaneously tolerating less than 20% Byzantine and less than 20% crash faults; however, this guarantee is impossible according to [52] and after careful analysis it is actually a 3δ protocol as Rotor requires 2δ and Votor an extra one. Optimistic fast paths have also been explored in synchronous and asynchronous settings [9, 15, 28, 36, 46, 51, 54].

For comparison, BlueBottle commits in two rounds under partial synchrony while tolerating up to f_c Byzantine faults with $n_c = 5f_c + 1$ replicas. Two-round finality with $n = 5f - 1$ replicas is achievable when the protocol explicitly identifies and ignores faulty leaders [7, 37]. BlueBottle could be extended with such leader-exclusion mechanisms to attain the $5f - 1$ bound; however, we deliberately avoid this design point because the incremental gain in fault tolerance is modest relative to the additional protocol design and engineering complexity.

Accountable safety [17, 45, 48] and accountable liveness [39, 57] strengthen classical safety and liveness via crypto-economic mechanisms. In addition to preserving agreement (safety) and eventual decision (liveness) below a fault threshold, these notions enable attribution of provable misbehavior, e.g., through slashing conditions, when safety or liveness is violated, thereby deterring equivocation or censorship in permissioned and permissionless settings.

Acknowledgments

This work is partially supported by Mysten Labs.

References

- [1] Ittai Abraham, Guy Gueta, Dahlia Malkhi, and Jean-Philippe Martin. 2018. Revisiting Fast Practical Byzantine Fault Tolerance: Thelma, Velma, and Zelma. *arXiv CS.DC 1801.10022*.
- [2] Ittai Abraham, Philipp Jovanovic, Mary Maller, Sarah Meiklejohn, and Gilad Stern. 2023. Bingo: Adaptivity and Asynchrony in Verifiable Secret Sharing and Distributed Key Generation. In *Advances in Cryptology – CRYPTO 2023*.
- [3] Ittai Abraham, Philipp Jovanovic, Mary Maller, Sarah Meiklejohn, Gilad Stern, and Alin Tomescu. 2023. Reaching Consensus for Asynchronous Distributed Key Generation. *Distributed Computing* 36 (2023), Issue 3.
- [4] Ittai Abraham, Dahlia Malkhi, Kartik Nayak, Ling Ren, and Maofan Yin. 2020. Sync HotStuff: Simple and Practical Synchronous State Machine Replication. In *2020 IEEE Symposium on Security and Privacy (SP '20)*.
- [5] Ittai Abraham, Kartik Nayak, Ling Ren, and Zhuolun Xiang. 2020. Byzantine Agreement, Broadcast and State Machine Replication with Near-optimal Good-case Latency. *arXiv cs.CR 2003.13155*.
- [6] Ittai Abraham, Kartik Nayak, Ling Ren, and Zhuolun Xiang. 2021. Good-case Latency of Byzantine Broadcast: A Complete Categorization. In *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing (PODC'21)*.
- [7] Ittai Abraham and Ling Ren. 2025. 2-round BFT in Simplex style for $n = 5f - 1$. <https://decentralizedthoughts.github.io/2025-08-06-5fminus1-simplex/>. <https://decentralizedthoughts.github.io/2025-08-06-5fminus1-simplex/>. Decentralized Thoughts. Accessed: 2025-11-13.
- [8] Balaji Arun, Zekun Li, Florian Suri-Payer, Sourav Das, and Alexander Spiegelman. 2024. Shoal++: High Throughput DAG BFT Can Be Fast! *arXiv preprint arXiv:2405.20488* (2024).
- [9] Pierre-Louis Aublin, Rachid Guerraoui, Nikola Knežević, Vivien Quéma, and Marko Vukolić. 2015. The Next 700 BFT Protocols. *ACM Trans. Comput. Syst.* 32, 4 (2015).
- [10] Kushal Babel, Andrey Chursin, George Danezis, Lefteris Kokoris-Kogias, and Alberto Sonnino. 2024. Mysticeti: Low-Latency DAG Consensus with Fast Commit Path. *arXiv:2310.14821 [cs.DC]*.
- [11] Renas Bacho and Julian Loss. 2022. On the Adaptive Security of the Threshold BLS Signature Scheme. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*.
- [12] Shehar Bano, Alberto Sonnino, Andrey Chursin, Dmitri Perelman, Zekun Li, Avery Ching, and Dahlia Malkhi. 2020. Twins: Bft systems made robust. *arXiv preprint arXiv:2004.10617* (2020).
- [13] Erica Blum, Jonathan Katz, Chen-Da Liu-Zhang, and Julian Loss. 2020. Asynchronous byzantine agreement with subquadratic communication. In *Theory of Cryptography: 18th International Conference, TCC 2020, Durham, NC, USA, November 16–19, 2020, Proceedings, Part I* 18. Springer, 353–380.
- [14] Dan Boneh, Ben Lynn, and Hovav Shacham. 2001. Short signatures from the Weil pairing. In *International conference on the theory and application of cryptology and information security*. Springer, 514–532.
- [15] Francisco V. Brasileiro, Fabiola Greve, Achour Mostéfaoui, and Michel Raynal. 2001. Consensus in One Communication Step. In *Proceedings of the 6th International Conference on Parallel Computing Technologies (PaCT '01)*. Springer-Verlag.
- [16] Ethan Buchman, Jae Kwon, and Zarko Milosevic. 2019. The latest gossip on BFT consensus. *arXiv cs.DC 1807.04938*.
- [17] Vitalik Buterin and Virgil Griffith. 2019. Casper the Friendly Finality Gadget. *arXiv cs.CR 1710.09437*.
- [18] Christian Cachin, Klaus Kursawe, and Victor Shoup. 2000. Random oracles in constant-time: practical asynchronous byzantine agreement using cryptography. In *Proceedings of the nineteenth annual ACM symposium on Principles of distributed computing*. 123–132.
- [19] Miguel Castro and Barbara Liskov. 1999. Practical Byzantine Fault Tolerance. In *Proceedings of the Third USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, New Orleans, Louisiana, USA, February 22–25, 1999, Margo I. Seltzer and Paul J. Leach (Eds.). USENIX Association, 173–186.
- [20] Benjamin Y Chan and Rafael Pass. 2023. Simplex Consensus: A Simple and Fast Consensus Protocol. In *Theory of Cryptography: 21st International Conference (TCC 2023)*. Springer-Verlag.
- [21] Brendan Kobayashi Chou, Andrew Lewis-Pye, and Patrick O’Grady. 2026. Min-immit: Fast Finality with Even Faster Blocks. *arXiv:2508.10862 [cs.DC]* <https://arxiv.org/abs/2508.10862>
- [22] Flaviu Cristian, Houtan Aghili, Ray Strong, and Danny Dolev. 1995. Atomic Broadcast: From Simple Message Diffusion to Byzantine Agreement. *Information and Computation, Volume 118, Issue 1* (1995).
- [23] George Danezis, Lefteris Kokoris-Kogias, Alberto Sonnino, and Alexander Spiegelman. 2022. Bridging the Gap of Timing Assumptions in Byzantine Consensus. In *EuroSys '22: Proceedings of the Seventeenth European Conference on Computer Systems*.
- [24] Henry de Valence. 2024. Ed25519 for consensus-critical contexts. <https://crates.io/crates/ed25519-consensus>.
- [25] Die.Net. 2024. writev(3) - Linux man page. <https://linux.die.net/man/3/writev>.
- [26] John R Douceur. 2002. The sybil attack. In *International workshop on peer-to-peer systems*. Springer, 251–260.
- [27] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. 1988. Consensus in the presence of partial synchrony. *Journal of the ACM (JACM)* 35, 2 (1988), 288–323.
- [28] Roy Friedman, Achour Mostéfaoui, and Michel Raynal. 2005. Simple and Efficient Oracle-Based Consensus Protocols for Asynchronous Byzantine Systems. *IEEE Transactions on Dependable and Secure Computing* 2, 1 (2005), 46–56.
- [29] Guy Golan Gueta, Ittai Abraham, Shelly Grossman, Dahlia Malkhi, Benny Pinkas, Michael Reiter, Dragos-Adrian Seredinschi, Orr Tamir, and Alin Tomescu. 2019. SBFT: A Scalable and Decentralized Trust Infrastructure. In *2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*.
- [30] Philipp Jovanovic, Lefteris Kokoris-Kogias, Bryan Kumara, Alberto Sonnino, Pasindu Tennage, and Igor Zablotchi. 2025. Mahi-Mahi: Low-Latency Asynchronous BFT DAG-Based Consensus. In *2025 IEEE 45th International Conference on Distributed Computing Systems (ICDS '25)*.
- [31] Idit Keidar, Eleftherios Kokoris-Kogias, Oded Naor, and Alexander Spiegelman. 2021. All You Need is DAG. In *PODC'21: Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing*.
- [32] Idit Keidar, Oded Naor, Ouri Poupko, and Ehud Shapiro. 2022. Cordial Miners: Fast and Efficient Consensus for Every Eventuality. In *37th International Symposium on Distributed Computing (DISC 2023)*.
- [33] Quentin Knip, Jakub Sliwinski, and Roger Wattenhofer. 2025. Alpenglow. <https://www.anza.xyz/alpenglow-1-1>.
- [34] Ramakrishna Kotla, Lorenzo Alvisi, Mike Dahlin, Allen Clement, and Edmund Wong. 2007. Zyzzyva: Speculative Byzantine Fault Tolerance. In *Proceedings of the 21st ACM SIGOPS Symposium on Operating Systems Principles (SOSP '07)*. Association for Computing Machinery.
- [35] Arber Kuçi, Christian Cachin, and Georgia Azzurra Marson. 2021. Proof-of-stake blockchain: Ouroboros.
- [36] Klaus Kursawe. 2002. Optimistic Byzantine Agreement. In *Proceedings of the 21st IEEE Symposium on Reliable Distributed Systems (SRDS '02)*. IEEE Computer Society.
- [37] Petr Kuznetsov, Andrei Tonkikh, and Yan X Zhang. 2021. Revisiting Optimal Resilience of Fast Byzantine Consensus. In *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing (PODC'21)*.
- [38] Mysten Labs. 2024. Mysticeti: Low-latency dag consensus with fast commit path. <https://github.com/asonnino/mysticeti>.
- [39] Andrew Lewis-Pye, Joachim Neu, Tim Roughgarden, and Luca Zanolini. 2025. Accountable Liveness. In *Proceedings of the 2025 on ACM SIGSAC Conference on Computer and Communications Security (CCS '25)*.
- [40] Julian Loss and Tal Moran. 2018. Combining asynchronous and synchronous byzantine agreement: The best of both worlds. *Cryptology ePrint Archive* (2018).
- [41] Dahlia Malkhi, Kartik Nayak, and Ling Ren. 2019. Flexible byzantine fault tolerance. In *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*. 1041–1053.
- [42] Jean-Philippe Martin and Lorenzo Alvisi. 2005. Fast Byzantine Consensus (FaB Paxos). In *Proceedings of the International Conference on Dependable Systems and Networks (DSN '05)*.
- [43] Meta. 2024. Sapling (Minibytes). <https://github.com/facebook/sapling/tree/main/eden/scm/lib/minibytes>.
- [44] Joachim Neu, Srivatsan Sridhar, Lei Yang, and David Tse. 2024. Optimal flexible consensus and its application to Ethereum. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 3885–3903.
- [45] Joachim Neu, Ertem Nusret Tas, and David Tse. 2022. The Availability-Accountability Dilemma and Its Resolution via Accountability Gadgets. In *Financial Cryptography and Data Security: 26th International Conference (FC '22)*.
- [46] Rafael Pass and Elaine Shi. 2018. Thunderella: Blockchains with Optimistic Instant Confirmation. In *Advances in Cryptology – EUROCRYPT 2018*. Springer International Publishing.
- [47] RustCrypto. 2024. RustCrypto: Hashes. <https://github.com/RustCrypto/hashes>.
- [48] Peiyao Sheng, Gerui Wang, Kartik Nayak, Sreeram Kannan, and Pramod Viswanath. 2021. BFT Protocol Forensics. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS '21)*. Association for Computing Machinery.
- [49] Victor Shoup. 2024. Sing a Song of Simplex. In *38th International Symposium on Distributed Computing, DISC 2024 (DISC '24)*.
- [50] Victor Shoup, Jakub Sliwinski, and Yann Vonlanthen. 2025. Kudzu: Fast and Simple High-Throughput BFT. *arXiv:2505.08771 [cs.DC]*
- [51] Nibesh Shrestha, Ittai Abraham, Ling Ren, and Kartik Nayak. 2020. On the Optimality of Optimistic Responsiveness. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS '20)*. Association for Computing Machinery.
- [52] Nibesh Shrestha, Aniket Kate, and Kartik Nayak. 2025. Hydrangea: Optimistic Two-Round Partial Synchrony. *Cryptology ePrint Archive, Paper 2025/1112*. <https://eprint.iacr.org/2025/1112>
- [53] Nibesh Shrestha, Rohan Shrothrium, Aniket Kate, and Kartik Nayak. 2024. Sailfish: Towards Improving Latency of DAG-based BFT. *Cryptology ePrint Archive* (2024).

- [54] Nibesh Shrestha, Qianyu Yu, Aniket Kate, Giuliano Losa, Kartik Nayak, and Xuechao Wang. 2025. Optimistic, Signature-Free Reliable Broadcast and Its Applications. arXiv:2505.02761 [cs.DC]
- [55] Alberto Sonnino. 2024. HotStuff. <https://github.com/asonnino/hotstuff>.
- [56] Alexander Spiegelman, Balaji Arun, Rati Gelashvili, and Zekun Li. 2023. Shoal: Improving dag-bft latency and robustness. *arXiv preprint arXiv:2306.03058* (2023).
- [57] Ertem Nusret Tas, David Tse, Fangyu Gai, Sreeram Kannan, Mohammad Ali Maddah-Ali, and Fisher Yu. 2023. Bitcoin-Enhanced Proof-of-Stake Security: Possibilities and Impossibilities. In *2023 IEEE Symposium on Security and Privacy (SP '23)*.
- [58] The Sui Team. 2025. Validator Deployment and Configuration. <https://docs.sui.io/guides/operator/validator/validator-config>.
- [59] The Tokio Team. 2024. Tokio. <https://tokio.rs>.
- [60] The Sui team. 2023. The Sui Blockchain. <http://sui.io>.
- [61] Gorgos Tsimos, Anastasios Kichidis, Alberto Sonnino, and Lefteris Kokoris-Gkias. 2024. Hammerhead: Leader reputation for dynamic scheduling. In *2024 IEEE 44th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 1377–1387.

A Open Science

We are committed to open science. All artifacts required to reproduce the results of this paper are available at anonymous hosting services for double-blind review:

- **Core protocol implementation** (BB-Core): <https://github.com/phvv/mysticeti/tree/odontoceti> (commit e02aeba)
- **Asynchronous variant** (BB-Core-Async): <https://github.com/phvv/mysticeti/tree/odontoceti-async> (commit 7b503fa)
- **Experimental dashboard and raw data**: <https://github.com/asonnino/mysticeti/blob/odontoceti/crates/orchestrator/assets/grafana-dashboard.json>

The repositories include the source code, build scripts, orchestration tooling, and configuration files used to run the experiments reported in Section 5, along with documentation for reproducing the benchmarks. No artifact is withheld. We provide a tutorial for reproducing the experiments in Appendix G. Upon acceptance, we will de-anonymize the repositories and release a tagged artifact for the ACM Artifact Evaluation process.

B Generative AI Usage

Various AI tools, such as Claude, Gemini, ChatGPT/Codex, and Grammarly, were used to assist with spell-checking, grammar, and general English writing. All changes made by the AI model were reviewed and approved by the authors. All content of the paper was written by the authors themselves, and the authors retain full responsibility for the accuracy, originality, and integrity of the work.

C Ethical Considerations

This work is a systems and protocol paper that does not involve human subjects, user data, personally identifiable information, or the exploitation of deployed systems. All experiments are run on validator nodes under our own control (cloud VMs) and do not interact with any third-party or production network. The protocols described here target permissionless and permissioned blockchain deployments where safety and liveness are the explicit goals; we see no realistic dual-use concerns beyond those common to consensus research. No responsible disclosure process was required.

D BB-Core Safety and Liveness

We prove that BB-Core implements Byzantine Atomic Broadcast (BAB) [22], which formalizes reliable message delivery with total ordering in the presence of Byzantine faults. In BAB, *broadcast* refers to a validator proposing a message for inclusion in the final sequence, while *deliver* refers to outputting a message from the total ordering for local processing. A BAB protocol must satisfy four properties:

- **Validity**: If an honest validator broadcasts a message m , then all honest validators eventually deliver m .
- **Agreement**: If an honest validator delivers a message m , then all honest validators eventually deliver m .
- **Integrity**: For any message m , an honest validator delivers m at most once, and only if m was previously broadcast by some validator.
- **Total Order**: If two honest validators both deliver messages m_1 and m_2 , they deliver them in the same order.

LEMMA 9. *There will never be a block that an honest validator directly commits while another honest validator directly skips.*

PROOF. Assume for the sake of contradiction that such a block exists and call it B . Thus, there are $4f_c + 1$ validators which support B and $4f_c + 1$ validators which blame B . Since f_c validators are Byzantine, there are $3f_c + 1$ honest validators that support B and a distinct set of $3f_c + 1$ honest validators that blame B . This means there are $7f_c + 2$ validators overall contradicting our assumption of $n_c = 5f_c + 1$. $\square$

LEMMA 10. *There will never be a block that an honest validator directly skips while another honest validator indirectly commits.*

PROOF. Assume for the sake of contradiction that such a block exists and call it B . Thus, there are $4f_c + 1$ validators which blame B and $2f_c + 1$ validators which support B . Since f_c validators in the network are Byzantine, there are $3f_c + 1$ honest validators that blame B and a distinct set of $f_c + 1$ honest validators that support B . This means there are $5f_c + 2$ validators overall contradicting our assumption of $n_c = 5f_c + 1$. $\square$

LEMMA 11. *If at a round R , $4f_c + 1$ blocks from distinct validators support a block B , then all blocks at future rounds $R' > R$ will link to $2f_c + 1$ supports for B from round R .*

PROOF. Each block links to $4f_c + 1$ blocks from the previous round. For the sake of contradiction, assume that a block B' in round $R' > R$ does not link to $2f_c + 1$ supports for B from round R . **Case:** $R' = R + 1$. B' should link to $4f_c + 1$ blocks from round R and also $4f_c + 1$ blocks in round R should support B . By quorum intersection, any two subsets of size $4f_c + 1$ from a set of size $5f_c + 1$ must share at least $3f_c + 1$ elements. Since f_c validators are Byzantine, the intersection contains at least $2f_c + 1$ correct validators. Thus, the only way for B' to not link to $2f_c + 1$ supports is if an honest validator equivocated in round R . This is a contradiction.

Case: $R' > R + 1$. B' links to $4f_c + 1$ blocks from round $R' - 1$. At least $3f_c + 1$ of these blocks are produced by honest validators. Honest validators always link to their own blocks, which means they will eventually link to their block from round $R + 1$. The above case proves how these blocks from round $R + 1$ link to $2f_c + 1$ supports

for B . Thus, the only way for B' to not link to $2f_c + 1$ supports for B is if all of these honest validators do not link to their own block in round $R + 1$. This is a contradiction. $\square$

As a result of Lemma 11, we have the following corollary.

COROLLARY 3. *There will never be a block which an honest validator directly commits while another honest validator indirectly skips.*

LEMMA 12. *All honest validators who have decided on a leader block, agree on the decision.*

PROOF. Let B_j and B_k be the highest committed leader blocks according to validators X and Y respectively. Without loss of generality, let $j \leq k$. Note that leader blocks decided by X which are higher than B_j are direct skips which according to Lemma 9 and Lemma 10 will be consistent with Y 's decision. The proof continues by induction on the statement for $0 \leq i \leq j$, if both X and Y decide on leader block B_i , then they either both commit or both skip the block.

Case: $i = j$. By definition, X directly commits B_i and from Lemma 9 and Corollary 3, Y will also commit B_i .

Case: Assuming the statement is true regarding B_i for $\ell + 1 \leq i \leq j$, we prove it is true for B_ℓ . This is done by enumerating decision possibilities.

- (1) If either validator directly commits B_ℓ , then by Lemma 9 and Corollary 3, the other will commit.
- (2) If either validator directly skips B_ℓ , then by Lemma 9 and Lemma 10, the other will skip.
- (3) Both X and Y indirectly decide B_ℓ . Let A_c^X and A_d^Y be the anchors used by X and Y to indirectly decide B_ℓ . Since $\ell + 1 < c \leq j$, it follows from the induction hypothesis that $A_c^X = A_d^Y$. Thus, both X and Y use the same anchor to decide B_ℓ . The indirect decision rule solely depends on the causal history of the anchor. By using the same anchor, X and Y will agree on the decision for B_ℓ . $\square$

THEOREM 1 (TOTAL ORDER). *BB-Core satisfies the total order property of BAB.*

PROOF. By Lemma 12, all honest validators agree on the decision for every leader block. The total ordering is a deterministic function of the sequence of committed leader blocks. Since all honest validators share the same sequence of committed leader blocks, they produce identical total orderings and deliver all blocks in the same order. $\square$

THEOREM 2 (INTEGRITY). *BB-Core satisfies the integrity property of BAB.*

PROOF. A block enters an honest validator's DAG only if it carries a valid signature from its author, so no unbroadcast block is ever delivered. The deterministic linearization processes each block as part of exactly one committed leader's causal history, so no honest validator delivers the same block twice. $\square$

LEMMA 13. *After GST, all honest validators will enter the same round within Δ .*

PROOF. Messages sent before GST will deliver in Δ after GST commences. Thus, the valid block of the highest round that any validator sent before GST will be delivered to all validators in GST + Δ . Upon receiving this block, all honest validators will enter the round. $\square$

LEMMA 14. *After GST, leader blocks from honest validators will receive support from all honest validators.*

PROOF. By Lemma 13, all honest validators will enter the same round within Δ after GST. When an honest validator sends its leader block for this round, it is delivered to all validators within Δ . Since the protocol sets the timeout to 2Δ , any honest validator that has entered the round will receive the leader block and cast support for it before timing out, regardless of when exactly they entered the round relative to other honest validators. $\square$

As a result of Lemma 14, we have the following corollary. Recall that there are $4f_c + 1$ honest validators.

COROLLARY 4. *After GST, leader blocks from honest validators will be (directly) committed.*

LEMMA 15. *The round-robin schedule of leader-block proposers ensures that, within a window of $2f_c + 2$ rounds, there are two consecutive rounds in which an honest validator is the proposer of the highest-ranked leader block.*

PROOF. The network contains f_c Byzantine validators. In $2f_c + 2$ rounds, there are $2f_c + 1$ sets of two consecutive rounds. Due to the schedule being round robin, in at least $f_c + 1$ of the rounds, an honest validator will be the proposer of the highest-ranked leader block. These blocks are the highest-ranked leader block in exactly two of the sets. By the pigeonhole principle, one set must contain $\lceil \frac{2 \cdot (f_c + 1)}{2f_c + 1} \rceil = 2$ honest validators proposing the highest-ranked leader block. $\square$

LEMMA 16. *After GST, undecided leader blocks will eventually be decided.*

PROOF. Let B be an undecided leader block in round r . By Lemma 15, after GST, there will be two consecutive rounds, j and $j + 1$ with $j > r$, where honest validators propose the highest-ranked leader block. By Corollary 4, their leader blocks will be committed. The proof continues by induction on the statement for rounds earlier than j , all leader blocks are decided.

Case: All undecided leader blocks in rounds $j - 1$ and $j - 2$ will be decided as they now have decided anchors in rounds $j + 1$ and j respectively.

Case: For undecided leader blocks in round $i < j - 2$, j is higher than the decision round of the wave that i is in. Thus, by the induction hypothesis, there are no undecided leader blocks between i and j . Hence, the leader block in round i will also be decided. $\square$

THEOREM 3 (VALIDITY). *BB-Core satisfies the validity property of BAB.*

PROOF. By Lemma 13, all honest validators synchronize to the same round within Δ after GST. By Lemma 15, within any window of $2f_c + 2$ rounds, there exist two consecutive rounds where honest

validators propose the highest-ranked leader blocks. By Corollary 4, these honest leader blocks will be directly committed. By Lemma 16, any previously undecided leader blocks will eventually be decided once we have committed blocks from honest validators in consecutive rounds. Since honest leader blocks are committed every $2f_c + 2$ rounds and each commitment resolves all pending undecided blocks, no block remains undelivered indefinitely. $\square$

THEOREM 4 (AGREEMENT). *BB-Core satisfies the agreement property of BAB.*

PROOF. Suppose an honest validator delivers a block B . Then B was delivered as part of the causal history of some committed leader block B' . By Lemma 12, all honest validators agree on the decision for B' . By Theorem 3, all honest validators eventually decide every leader block. Therefore, all honest validators eventually commit B' and deliver B . $\square$

D.1 Checkpoint Safety

We now prove the safety of the checkpoint path under the extended threat model that includes alive-but-corrupt (AbC) validators.

LEMMA 17 (UNIQUE CHECKPOINTQC PER HEIGHT). *Let $n_c = 5f_c + 1$ and $q = 4f_c + 1$. If the total number of equivocating validators (Byzantine plus AbC) satisfies $f_c + f_{AbC} \leq 3f_c$, then no two conflicting CheckpointQCs can be formed for the same slot height s .*

PROOF. By the protocol (Section 3.4), an honest validator broadcasts at most one checkpoint proposal per slot height. Suppose, for contradiction, that two CheckpointQCs C_1 and C_2 exist for the same height s but for different state roots ($\sigma_1 \neq \sigma_2$). Each requires $q = 4f_c + 1$ signatures from distinct validators. Since honest validators do not sign conflicting checkpoint proposals at the same height, any validator appearing in both signer sets must be equivocating. The minimum overlap between the two signer sets is $2q - n_c = 2(4f_c + 1) - (5f_c + 1) = 3f_c + 1$. Thus, at least $3f_c + 1$ validators must equivocate. This contradicts our assumption that the total number of equivocating validators is at most $f_c + f_{AbC} \leq 3f_c < 3f_c + 1$. $\square$

THEOREM 5 (CHECKPOINT-PATH SAFETY). *If two clients finalize heights via the checkpoint path by observing FinalityQCs, they will never commit conflicting states for the same slot height, provided $f_c + f_{AbC} \leq 3f_c$.*

PROOF. A client finalizes a state for height s only upon observing a FinalityQC, which requires an underlying valid CheckpointQC. By Lemma 17, there is at most one CheckpointQC per height s . Since honest validators only broadcast a checkpoint witness matching the uniquely determined CheckpointQC (and only if its payload executes to the correct state), no FinalityQC can be formed for a conflicting state.

During fork recovery (Section 4), validators exchange all known CheckpointQCs via Byzantine Broadcast. Because Lemma 17 guarantees at most one CheckpointQC exists per height, the sequence of CheckpointQCs forms a single non-conflicting chain. All correct validators deterministically adopt the branch belonging to the CheckpointQC with the highest slot, preserving any state finalized by a checkpoint-path client prior to the fork. $\square$

E Implementation and Evaluation Details

E.1 Implementation

We implement a networked, multi-core BB-Core validator in Rust by forking the Mysticeti codebase [38]. Our implementation leverages `tokio` [59] for asynchronous networking, utilizing raw TCP sockets for communication without relying on any RPC frameworks. For cryptographic operations, we rely on `ed25519-consensus` [24] for asymmetric cryptography and `blake2` [47] for cryptographic hashing. To ensure data persistence and crash recovery, we employed the Write-Ahead Log (WAL). Our WAL optimizes I/O operations through vectored writes [25] and efficient memory-mapped file usage with the `minibytes` [43] crate, minimizing data copying and serialization.

In addition to regular unit tests, we inherited and utilized two supplementary testing utilities from the Mysticeti codebase. First, a simulation layer replicates the functionality of the `tokio` runtime and TCP networking. This simulated network accurately simulates real-world WAN latencies, while the `tokio` runtime simulator employs a discrete event simulation approach to mimic the passage of time. Second, a command-line utility (called *orchestrator*) which deploys real-world clusters of BB-Core on machines distributed across the globe.

We are open-sourcing our BB-Core implementation, along with its simulator and orchestration tools, to ensure reproducibility of our results⁴.

E.2 Experimental Setup

This section complements Section 5 by detailing the experimental setup used to evaluate BB-Core.

We deploy BB-Core on AWS, using `m5d.8xlarge` instances across 13 different AWS regions: Northern Virginia (us-east-1), Oregon (us-west-2), Canada (ca-central-1), Frankfurt (eu-central-1), Ireland (eu-west-1), London (eu-west-2), Paris (eu-west-3), Stockholm (eu-north-1), Mumbai (ap-south-1), Singapore (ap-southeast-1), Sydney (ap-southeast-2), Tokyo (ap-northeast-1), and Seoul (ap-northeast-2). Validators are distributed across those regions as equally as possible. Each machine provides 10 Gbps of bandwidth, 32 virtual CPUs (16 physical cores) on a 3.1 GHz Intel Xeon Skylake 8175M, 128 GB memory, and runs Linux Ubuntu server 24.04. We select these machines because they provide decent performance, are in the price range of “commodity servers”, and match the minimal specifications of modern quorum-based blockchains [58].

In Section 5, *latency* refers to the time elapsed from the moment a client submits a transaction to when it is committed by the validators, and *throughput* refers to the number of transactions committed per second. We instantiate several geo-distributed benchmark clients within each validator submitting transactions in an open loop model, at a fixed rate. We experimentally increase the load of transactions sent to the systems, and record the throughput and latency of commits. As a result, all plots Section 5 illustrate the steady-state latency of all systems under low load, as well as the maximal throughput they can provide after which latency grows quickly. Transactions in the benchmarks are arbitrary and contain

⁴ <https://github.com/phvv/mysticeti/tree/odontoceti> (commit e02aeba)

512 bytes. We configure both BB-Core and Mysticeti with 2 leaders per round, and a leader timeout of 1 second.

F Asynchronous BB-Core

We present BB-Core-Async, the asynchronous variant of BB-Core. It builds upon the same ideas as BB-Core, but operates in a fully asynchronous network model and leverages a threshold common coin to achieve liveness. BB-Core-Async is the first $5f + 1$ -validator BFT consensus protocol to achieve both safety and liveness in a fully asynchronous network. It achieves low latency through its shortened commit path. BB-Core-Async algorithm is Algorithm 1, Algorithm 2, and Algorithm 3 when we also include the orange-colored lines.

F.1 Additional Assumptions

First, the communication network is asynchronous and messages can be delayed arbitrarily, but messages among honest validators are eventually delivered.

Additionally, we employ a global perfect coin to introduce randomization, similar to previous work [13, 18, 31, 40]. This coin can be constructed using an adaptively secure threshold signature scheme [11, 14], with the distributed key setup performed under asynchronous conditions [2, 3].

As with BB-Core (proven in Appendix D of the appendix), BB-Core-Async implements BAB [22].

All notation in the proofs below refers to the core layer; for clarity, we drop the subscript and write f and n in place of f_c and n_c .

F.2 Safety Lemmas

We start by proving the Total Order and Integrity properties of BAB. A crucial intermediate result towards these properties is that all honest validators have consistent commit sequences, i.e., the committed sequence of one honest validator is a prefix of another's, or vice-versa. This is shown in Lemma 22 and Lemma 23, which the following lemmas and observations build up to.

DEFINITION 1 (WEAK CERTIFICATE). A weak certificate for a block b in round r is a set of $2f + 1$ distinct valid round- $(r + 2)$ blocks, each from a distinct validator, that support b .

DEFINITION 2 (STRONG CERTIFICATE). A strong certificate for a block b in round r is a set of $4f + 1$ distinct valid round- $(r + 2)$ blocks, each from a distinct validator, that support b .

LEMMA 18 (STRONG-TO-WEAK PROPAGATION). Let b be a round- r block with a strong certificate S (i.e., $|S| = 4f + 1$ distinct round- $(r + 2)$ blocks supporting b , Definition 2). Then every valid block at any future round $r' \geq r + 3$ has paths to at least $2f + 1$ blocks from S ; equivalently, it indirectly references a weak certificate for b (Definition 1).

PROOF. We prove the claim by induction on r' , starting at $r' = r + 3$.

Base case ($r' = r + 3$): Let x be a valid round- $(r + 3)$ block. By construction, x references $4f + 1$ distinct valid round- $(r + 2)$ blocks from distinct validators; thus at most f of these can be Byzantine, so x includes at least $3f + 1$ round- $(r + 2)$ blocks from

honest validators. Likewise, the strong certificate S contains at least $3f + 1$ blocks from honest validators (since at most f of its $4f + 1$ members can be Byzantine). Restricting attention to honest validator identities (a universe of size $4f + 1$), these two sets intersect in at least $(3f + 1) + (3f + 1) - (4f + 1) = 2f + 1$ blocks. Hence x directly references at least $2f + 1$ members of S , so x has paths to a weak certificate for b .

Induction step: Assume that every valid round- r' block, for some $r' \geq r + 3$, has paths to at least $2f + 1$ members of S . Consider any valid round- $(r' + 1)$ block y . By construction, y references $4f + 1$ distinct round- r' blocks. By the induction hypothesis, each of those round- r' blocks has paths to at least $2f + 1$ members of S . Therefore, y references at least one round- r' block that has paths to at least $2f + 1$ members of S ; by transitivity of paths, y itself has paths to at least $2f + 1$ members of S , i.e., to a weak certificate for b . $\square$

OBSERVATION 1. A block cannot support for more than one block proposal from a given validator, in a given round.

PROOF. This is by construction. Honest validators interpret support in the DAG through deterministic depth-first traversal. So even if a block b in the vote round has paths to multiple leader round blocks from the same validator v (i.e., equivocating blocks), all honest validators will interpret b to vote for only one of v 's blocks (the first block to appear in the depth-first traversal starting from b). $\square$

LEMMA 19 (STRONG CERTIFICATE EXCLUSIVITY). Fix a validator v and round r . If some round- r block b of v has a strong certificate, then no other round- r block $b' \neq b$ of v can have even a weak certificate. In particular, at most one round- r block of v can have a strong certificate.

PROOF. Suppose, for contradiction, that b has a strong certificate S (so $|S| = 4f + 1$ decision-round blocks at round $r + 2$ support b), and some other block $b' \neq b$ from the same validator and round has a weak certificate W (so $|W| = 2f + 1$ vote-round blocks at round $r + 2$ support b'). Both S and W are over the universe of $5f + 1$ validator identities and contain one valid block per identity.

Let $\text{Id}(S)$ and $\text{Id}(W)$ be the sets of validator identities appearing in the certificates S and W (one valid round- $(r + 2)$ block per identity). Then $|\text{Id}(S)| = 4f + 1$ and $|\text{Id}(W)| = 2f + 1$. By quorum intersection over identities,

$$|\text{Id}(S) \cap \text{Id}(W)| \geq (4f + 1) + (2f + 1) - (5f + 1) = f + 1 > 0.$$

Since at most f validators are Byzantine overall, this intersection contains at least one honest identity. Pick such an honest validator $u \in \text{Id}(S) \cap \text{Id}(W)$, and let x be u 's unique valid decision-round (round $r + 2$) block (by the uniqueness-per-round assumption). Because certificates record one valid block per identity, the same block x is the representative of u in both S and W , hence x supports both b and b' . This contradicts Observation 1. Therefore, b' cannot have a weak certificate. Taking W to be a strong certificate S' gives the special case that two distinct blocks of v in the same round cannot both have strong certificates. $\square$

OBSERVATION 2. If an honest validator v directly or indirectly commits a block b , then v 's local DAG contains a weak certificate for b .

PROOF. This follows immediately from our direct and indirect commit rules. $\square$

OBSERVATION 3. *Honest validators agree on the sequence of leader slots.*

PROOF. This follows immediately from the properties of the common coin, see Appendix F.1. $\square$

OBSERVATION 4 (UNIQUE VALID BLOCK PER (VALIDATOR, ROUND)). *For any validator w and round r , at most one block signed by w is counted as valid in round r . Since commit rules apply only to valid blocks, any commitment for a slot with leader w in round r must be that unique valid block.*

PROOF. Immediate from the DAG construction and the fact that commit rules consider only valid blocks. $\square$

LEMMA 20. *If an honest validator v commits some block b in a slot s , then no other honest validator decides to directly skip the slot s .*

PROOF. Assume by contradiction that some honest validator v' decides to directly skip s . By the direct-skip rule, this means that in v' 's local DAG there exists a set N of $4f + 1$ distinct valid round- $(r + 2)$ blocks (one per validator identity) whose support does not go to b (a direct-skip witness for s).

Since v commits b at s , by Observation 2 there exists a weak certificate W for b at s in v 's local DAG: a set of $2f + 1$ distinct valid round- $(r + 2)$ blocks (one per validator identity) that support b (Definition 1). Consider the identity sets $\text{Id}(N)$ and $\text{Id}(W)$. We have $|\text{Id}(N)| = 4f + 1$ and $|\text{Id}(W)| = 2f + 1$, hence by quorum intersection over the $5f + 1$ validator identities,

$$|\text{Id}(N) \cap \text{Id}(W)| \geq (4f + 1) + (2f + 1) - (5f + 1) = f + 1.$$

As at most f validators are Byzantine, the intersection contains at least one honest identity. Pick such an honest validator $u \in \text{Id}(N) \cap \text{Id}(W)$, and let x be u 's unique valid round- $(r + 2)$ block.

Because certificates/witnesses record one valid block per identity, the same block x is the representative of u in both N and W . By definition of W , x supports b ; by definition of N , the very same x is counted as not supporting b at v' . By Observation 1 (uniqueness of support per validator/round under the deterministic rule), x cannot both support and not support b . Hence v' cannot have a valid direct-skip witness N , a contradiction. $\square$

LEMMA 21. *If an honest validator directly commits some block in a slot s , then no other honest validator decides to skip the slot s .*

PROOF. Assume by contradiction that an honest validator v directly commits block b in slot s while another honest validator v' decides to skip s . By Lemma 20, v' cannot directly skip s ; therefore v' must attempt to skip s via the indirect decision rule. Let r be the round of s .

Since v directly commits b , there exists a strong certificate S for b at s (i.e., $|S| = 4f + 1$ distinct valid round- $(r + 2)$ blocks supporting b). By Lemma 18 (Strong-to-Weak Propagation), every valid block at any round $r' \geq r + 3$ has paths to at least $2f + 1$ members of S , i.e., it carries a weak certificate for b . In particular, any valid anchor block for deciding s (which necessarily lies at some round $r' \geq r + 3$) has paths to a weak certificate for b .

But the indirect skip rule for s requires an anchor whose deterministic support excludes b (equivalently, with no weak certificate

for b). This contradicts the propagation property above. Hence v' cannot skip s indirectly either. Contradiction. $\square$

LEMMA 22. *If a slot s is committed at two honest validators, then s contains the same block at both validators.*

PROOF. Let v and u be honest validators and suppose v commits block b at slot s . If u commits s as well, we show that u commits b .

Let w be the validator identity of the leader for s (i.e., the creator of b). By Observation 3, all honest validators agree on the leader identity per slot, so u also agrees that s must contain a block by w .

By Observation 4, there is a unique valid round- r block by w that can be committed in s .

Since v commits b and b is by w , b is this unique valid block by w for round r . Hence if u commits s , it must also commit b . Thus s contains the same block at both validators. $\square$

We say that a slot is *decided* at a validator v if s is committed or skipped, that is, if it is categorized as *commit* or *skip*. Otherwise, s is *undecided*.

LEMMA 23. *If a slot s is decided at two honest validators v and v' , then either both validators commit s , or both validators skip s .*

PROOF. Assume by contradiction that there exists a slot s such that v and v' decide differently at s . We consider a finite execution prefix and assume wlog that s is the highest slot at which v and v' decide differently (*). Further assume wlog that v commits s and v' skips s . By Lemma 20 and Lemma 21, neither v nor v' could have used the direct decision rule for s ; they must both have used the indirect rule. Consider now the anchor of s : v and v' must agree on which slot is the anchor of s , since by our assumption (*) above, they make the same decisions for all slots higher than s , including the anchor of s . Let s' be the anchor of s ; s' must be committed at both v and v' . Thus, by Lemma 22, v and v' commit the same block b' at s' . But then v and v' cannot reach different decisions about slot s using the indirect decision rule. We have reached a contradiction. $\square$

We have proven the consistency of honest validators' commit sequences: honest validators commit (or skip) the same leader blocks, in the same order. However, we are not done: we also need to prove that non-leader blocks are delivered in the same order by honest validators. We show this next.

Causal history and delivery conditions Consider an honest validator v . We call the *causal history* of a block b in v 's DAG, the transitive closure of all blocks referenced by b in v 's DAG, including b itself. In BB-Core-Async, a block b is delivered by an honest validator v if (1) there exists a committed leader block l in v 's DAG such that b is in l 's causal history (2) all slots up to l are decided in v 's DAG and (3) b has not been delivered as part of a lower slot's causal history. In this case we say b is *delivered* at slot s , or *delivered with* block l .

LEMMA 24. *If a block b is delivered by two honest validators v and v' , then b is delivered at the same slot s , and b is delivered with the same leader block l , at both v and v' .*

PROOF. Let s be the slot at which b is delivered at validator v , and l the corresponding leader block in s , also at validator v . Consider

now the slot s' at which b is delivered at validator v' , and l' the corresponding leader block. Assume by contradiction that $s' \neq s$. If $s' < s$, then v would have also delivered b at slot s' , since by Lemma 22 must commit the same leader blocks in the same slots, so v could not have delivered b again at slot s ; a contradiction. Similarly, if $s < s'$, then v' would have already delivered b at slot s , since by Lemma 22 v and v' must have committed the same block in slot s ; contradiction. Thus it must be that $s = s'$, and by Lemma 22, $l = l'$. $\square$

We can now prove the main safety properties of BlueBottle: Total Order and Integrity.

THEOREM 6 (TOTAL ORDER). *BB-Core-Async satisfies the total order property of Byzantine Atomic Broadcast.*

PROOF. This property follows immediately from Lemma 24 and the fact that honest validators order the causal histories of committed blocks using the same deterministic function, and deliver blocks in this order. $\square$

THEOREM 7 (INTEGRITY). *BB-Core-Async satisfies the integrity property of Byzantine Atomic Broadcast.*

PROOF. This is by construction: a block b is delivered as part of the causal history of a committed leader block only if b has not been delivered along with an earlier leader block (see "Causal history & delivery conditions" above). So an honest validator cannot deliver the same block twice. $\square$

F.3 Liveness Lemmas

Block inclusion. The following two lemmas establish that blocks broadcast by honest validators are eventually included in all honest validators' DAGs.

LEMMA 25. *If a block b produced by an honest validator v references some block b' , then b' will eventually be included in the local DAG of every honest validator.*

PROOF. This is ensured by the synchronizer sub-component in each validator: if some validator w receives b from v , but does not have b' yet, w will request b' from v ; since v is honest and the network links are reliable, v will eventually receive w 's request, send b' to w , and w will eventually receive b' . The same is recursively true for any blocks from the causal history of b' , so w will eventually receive all blocks from the causal history of b' and thus include b' in its local DAG. $\square$

LEMMA 26. *If a honest validator v broadcasts a block b , then b will eventually be included in the local DAG of every honest validator.*

PROOF. Since network links are reliable, all honest validators will eventually receive b from v . By Lemma 25, all honest validators will eventually receive all of b 's causal history, and so will include b in their local DAG. $\square$

Main structural lemmas. These are the main structural lemmas that we will use to prove liveness. The key idea is that the reference rule creates significant overlap among honest blocks across consecutive rounds, which we can leverage to ensure that a randomly chosen leader has sufficient honest support to be directly committed.

LEMMA 27 (REFERENCE HONESTY LOWER BOUND). *Any valid round- $(R + 1)$ block (whether created by an honest or Byzantine validator) references at least $3f + 1$ honest round- R blocks.*

PROOF. In any round there are $5f + 1$ total validators of which at most f are Byzantine, so at least $4f + 1$ are honest. A valid round- $(R + 1)$ block, by definition of the protocol's formation rule, references at least $4f + 1$ distinct round- R blocks (one per validator identity). At most f of these can be Byzantine, hence at least $4f + 1 - f = 3f + 1$ are honest. $\square$

LEMMA 28 (MANY HEAVILY REFERENCED ROUND- R BLOCKS). *There are at least $2f + 1$ honest round- R blocks each referenced by at least $f + 1$ honest round- $R + 1$ blocks.*

PROOF. By Lemma 27 every valid round- $(R + 1)$ block references at least $3f + 1$ honest round- R blocks. There are $4f + 1$ honest validators, hence $4f + 1$ honest round- $(R + 1)$ blocks in total. Consider the bipartite graph whose left vertices A are the honest round- $(R + 1)$ blocks and whose right vertices B are the honest round- R blocks; connect $a \in A$ to $b \in B$ if a references b .

Let E be the total number of edges. Each $a \in A$ has degree at least $3f + 1$, so

$$E \geq (4f + 1)(3f + 1).$$

Suppose for contradiction that fewer than $2f + 1$ honest round- R blocks have degree at least $f + 1$ (i.e., are referenced by $\geq f + 1$ honest round- $(R + 1)$ blocks). Let $X \subseteq B$ be the (assumed) set of degree $\geq f + 1$ blocks with $|X| \leq 2f$. Any block in $B \setminus X$ then has degree at most f .

We upper bound E under this hypothesis:

$$\begin{aligned} E &\leq |X|(4f + 1) + (|B| - |X|)f \\ &= |X|(4f + 1 - f) + f(4f + 1) \\ &= |X|(3f + 1) + f(4f + 1). \end{aligned}$$

Using $|X| \leq 2f$ we get

$$\begin{aligned} E &\leq 2f(3f + 1) + f(4f + 1) \\ &= (6f^2 + 2f) + (4f^2 + f) \\ &= 10f^2 + 3f. \end{aligned}$$

Yet

$$(4f + 1)(3f + 1) = 12f^2 + 7f + 1 > 10f^2 + 3f,$$

a contradiction. Therefore, $|X| \geq 2f + 1$, establishing the claim. $\square$

LEMMA 29 (COMMON HONEST ANCESTORS). *Any set of $4f + 1$ round- $R + 2$ blocks collectively references at least $2f + 1$ common honest round- R blocks.*

PROOF. Let H denote the set of (at least) $2f + 1$ honest round- R blocks guaranteed by Lemma 28: every $h \in H$ is referenced by at least $f + 1$ honest round- $(R + 1)$ blocks.

Consider any multiset S of $4f + 1$ round- $(R + 2)$ blocks (they may be arbitrary, honest or Byzantine). By applying Lemma 27 to round $R + 1$ vs. $R + 2$, each valid round- $(R + 2)$ block includes at least $3f + 1$ honest round- $(R + 1)$ blocks among the $4f + 1$ distinct references it must carry.

Fix some $h \in H$. Suppose, for contradiction, that a particular $s \in S$ fails to (indirectly) reference h . Then s must omit every honest round- $(R + 1)$ block that references h . However, h has at least $f + 1$ such honest round- $(R + 1)$ children, while s can omit at most $(4f + 1) - (3f + 1) = f$ honest round- $(R + 1)$ blocks (because it necessarily includes at least $3f + 1$ of the $4f + 1$ honest ones). Since $f + 1 > f$, omitting them all is impossible. Therefore, every $s \in S$ (indirectly) references h .

The argument holds for each $h \in H$, so all blocks in S commonly (indirectly) reference every element of H . Thus, their intersection over round- R ancestors contains H , and has size at least $|H| \geq 2f + 1$. $\square$

Liveness theorems. We now leverage the structural overlap to obtain probabilistic liveness via the common coin.

DEFINITION 3 (CORE SET OF ROUND R). Let C_R be the set of honest round- R blocks each referenced by at least $f + 1$ honest round- $(R + 1)$ blocks. By Lemma 28, $|C_R| \geq 2f + 1$.

LEMMA 30 (PERSISTENCE OF CORE SUPPORT). Every valid round- $(R + 2)$ block (indirectly) references every block in C_R .

PROOF. Immediate from Lemma 29 since $C_R \subseteq H$ for the set H used there. $\square$

We denote by $l \leq 5f + 1$ the number of leader slots per round.

LEMMA 31 (DIRECT COMMITMENT VIA CORE INTERSECTION). Fix a round r and let $n = 5f + 1$. When l leader slots are sampled uniformly at random without replacement from the n validators, the probability that at least one slot can be directly committed is

$$p^* = 1 - \frac{\binom{n - |C_R|}{l}}{\binom{n}{l}}.$$

Moreover, if $l > n - |C_R|$ (in particular, if $l > 3f$ using $|C_R| \geq 2f + 1$) then $p^* = 1$ (deterministic success).

PROOF. By Lemma 30, any selected leader whose block lies in C_R is (eventually) directly commit-able by every honest validator. Thus a successful direct commitment in round r occurs iff the sampled set intersects C_R . The probability that it does not intersect C_R is exactly the hypergeometric zero-success probability $\binom{n - |C_R|}{l} / \binom{n}{l}$; subtracting from 1 yields p^* .

If $l > n - |C_R|$, it is impossible to choose all l leaders outside C_R , so the intersection is certain and $p^* = 1$. Using only $|C_R| \geq 2f + 1$, we have $n - |C_R| \leq 5f + 1 - (2f + 1) = 3f$, hence any $l > 3f$ suffices for determinism. $\square$

LEMMA 32 (EVENTUAL SLOT RESOLUTION). Fix a slot s . Every honest validator eventually either commits or skips s , with probability 1.

PROOF. We prove the lemma by showing that the probability of s remaining undecided forever at some honest validator is 0. In

order for s to remain undecided forever, s cannot be committed or skipped directly. Furthermore, s cannot be decided using the indirect rule. This means that the anchor s' of s must also remain undecided forever, and therefore the anchor s'' of s' must remain undecided forever, and so on. The probability of this occurring is at most equal to the probability of an infinite sequence of rounds with no directly committed slots, equal to $\lim_{t \rightarrow \infty} (1 - p^*)^t = 0$, where $p^* > 0$ is the probability from Lemma 31. $\square$

THEOREM 8 (VALIDITY). *BB-Core-Async satisfies the validity property of Byzantine Atomic Broadcast.*

PROOF. Let v be an honest validator and b a block broadcast by v . We show that, with probability 1, b is eventually delivered by every honest validator. Lemma 26 b is eventually included in the local DAG of every honest validator. So every honest validator will eventually include a reference to b in at least one of its blocks. Let r be the highest round at which some honest validator includes a reference to b in one of its blocks. By Lemma 31, with probability 1, eventually some block b' at a round $r' > r$ will be directly committed. Block b' must reference at least $4f + 1$ blocks, thus at least $3f + 1$ blocks from honest validators. Since all validators have b in their causal histories by round r , b' must therefore have a path to b . Lemma 32 guarantees that all slots before b' are eventually decided, so b' is eventually delivered. Thus, b will be delivered at all honest validators at the latest when b' is delivered along with its causal history. $\square$

THEOREM 9 (AGREEMENT). *BB-Core-Async satisfies the agreement property of Byzantine Atomic Broadcast.*

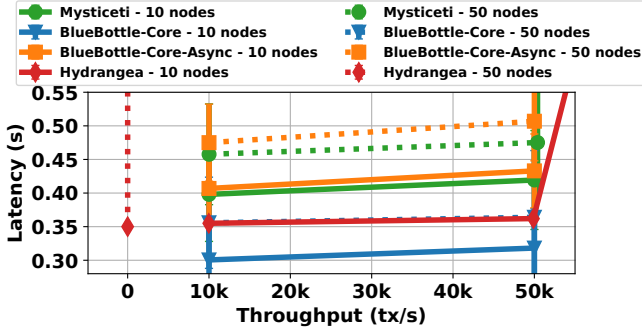
PROOF. Let v be an honest validator and b a block delivered by v . We show that, with probability 1, b is eventually delivered by every honest validator. Let l be the leader block with which b is delivered, and s the corresponding slot. By Lemma 32, all blocks up to and including s are eventually decided by all honest validators, with probability 1. By Lemma 24, all honest validators commit l in s . Therefore, all honest validators deliver b eventually. $\square$

F.4 BB-Core-Async Performance

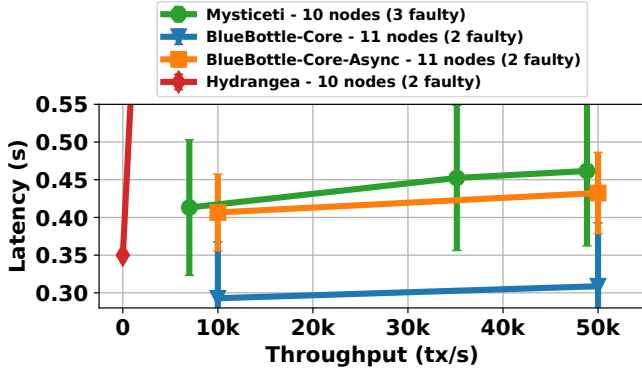
We implement⁵ BB-Core-Async by extending our BB-Core prototype (Appendix E.1) and evaluate it using the same setup as in Section 5.

Figure 4 presents the WAN throughput-latency results for BB-Core and BB-Core-Async, with Mysticeti and Hydrangea included as visual reference points from Section 5. We consider two scenarios: (a) committees of 10 and 50 validators without faults (Figure 4a), and (b) committees of 10 and 11 validators (enough to tolerate 3 and 2 faults) under corresponding crash failures (Figure 4b). We cap throughput at 50,000 tps for cost reasons. BB-Core-Async matches Mysticeti's throughput-latency performance across both scenarios and committee sizes, as both commit in three rounds in the common case. This highlights a key trade-off: for equivalent performance, one can choose between an asynchronous protocol with $n = 5f + 1$ or a partially synchronous one with $n = 3f + 1$.

⁵ <https://github.com/phvv/mysticeti/tree/odontoceti-async> (commit 7b503fa)



(a) Committees with 10 and 50 validators with no validator faults.



(b) With 3 or 2 crash faults; committees with 10 and 11 validators (minimum to tolerate 3 or 2 faults).

Figure 4: WAN throughput-latency performance comparison of BB-Core ($n = 5f + 1$) and BB-Core-Async ($n = 5f + 1$), with Mysticeti ($n = 3f + 1$) and Hydrangea ($n = 3f + 2c + k + 1$) as reference points. Transaction size is 512 B. The y-axis starts at 300 ms to zoom in on the latency difference between the systems.

G Reproducing Experiments

Below, we provide a tutorial for reproducing the experiments. All artifacts necessary for evaluating the contributions of this paper are publicly available at:

<https://github.com/phvv/mysticeti/tree/odontoceti> (commit e02aeba)

We provide the orchestration scripts used to benchmark the BlueBottle codebase on AWS and produce the benchmarks of Section 5.

Deploying a testbed. The file ‘`~/aws/credentials`’ should have the following content:

```
[default]
aws_access_key_id = YOUR_ACCESS_KEY_ID
aws_secret_access_key = YOUR_SECRET_ACCESS_KEY
```

configured with account-specific AWS *access key id* and *secret access key*. It is advise to not specify any AWS region as the orchestration scripts need to handle multiple regions programmatically.

A file ‘`settings.yaml`’ contains all the configuration parameters for the testbed deployment. We run the experiments of Section 5 with the following settings:

```
testbed_id: "${USER}-bluebottle-testbed"
cloud_provider: aws
token_file: "/Users/${USER}/.aws/credentials"
ssh_private_key_file: "/Users/${USER}/.ssh/aws"
regions:
  - us-east-1
  - us-west-2
  - ca-central-1
  - eu-central-1
  - eu-west-1
  - eu-west-2
  - eu-west-3
  - eu-north-1
  - ap-south-1
  - ap-southeast-1
  - ap-southeast-2
  - ap-northeast-1
  - ap-northeast-2
specs: m5d.8xlarge
repository:
  url: https://github.com/AUTHOR/REPO.git
  commit: main
node_parameters_path:
  "crates/orchestrator/assets/node-parameters.yml"
client_parameters_path:
  "crates/orchestrator/assets/client-parameters.yml"
benchmark_duration: 1000
```

where the file ‘`/Users/$USER/.ssh/aws`’ holds the ssh private key used to access the AWS instances, and ‘`AUTHOR`’ and ‘`REPO`’ are respectively the GitHub username and repository name of the codebase to benchmark. To run benchmarks with faults (Section 5.2), we add the following configuration at the end of the ‘`settings.yaml`’ file:

```
faults: !Permanent { faults: 3 }
```

The orchestrator binary provides various functionalities for creating, starting, stopping, and destroying instances. For instance, the following command boots 2 instances per region (if the settings file specifies 13 regions, as shown in the example above, a total of 26 instances will be created):

```
cargo run --bin orchestrator -- testbed deploy --instances 2
```

The following command displays the current status of the testbed instances

```
cargo run --bin orchestrator testbed status
```

Instances listed with a green number are available and ready for use and instances listed with a red number are stopped. It is necessary to boot at least one instance per load generator, one instance per validator, and one additional instance for monitoring purposes (see below). The following commands respectively start and stop instances:

```
cargo run --bin orchestrator -- testbed start
cargo run --bin orchestrator -- testbed stop
```

It is advised to always stop machines when unused to avoid incurring in unnecessary costs.

Running Benchmarks. Running benchmarks involves installing the specified version of the codebase on all remote machines and running one validator and one load generator per instance. For example, the following command benchmarks a committee of 50 validators under a constant load of 1,000 tx/s:

```
cargo run --bin orchestrator -- benchmark \
  --committee 50 fixed-load --loads 1000
```

The nodes and clients configuration files (respectively specified in ‘`crates/orchestrator/assets/node-parameters.yml`’ and then ‘`crates/orchestrator/assets/client-parameters.yml`’) are used to respectively parametrize the nodes and clients. We run our benchmarks with the nodes configuration file as follows:

```
leader_timeout:
  secs: 1
  nanos: 0
```

and the client configuration file as follows:

```
initial_delay:  
  secs: 400  
  nanos: 0
```

Monitoring. The orchestrator provides facilities to monitor metrics. It deploys a Prometheus instance and a Grafana instance on a

dedicated remote machine. Grafana is then available on the address printed on `stdout` when running benchmarks with the default username and password both set to `admin`. An example Grafana dashboard can be found in the file `'grafana-dashboard.json'`⁶.

⁶ <https://github.com/asonnino/mysticeti/blob/odontoceti/crates/orchestrator/assets/grafana-dashboard.json>